

Proxmox Storage DRS -- Operator Manual

Installing, configuring and running it safely

Bernd Zeimetz bernd@bzed.de

2026-09-23

Rendered from docs/manual/*.md, SHA-256 903f1f2ae3ee99d2dbb47eb7c41d1ab0c5f214812dec3aabab7d04deb78ddcaa
(sha256sum --check docs/pve-storage-drs-manual.pdf.sha256 in the repository must succeed).

Copyright © 2026 Bernd Zeimetz. Licensed under the GNU Affero General Public License, version 3 or later.

Contents

What you need	7
Setting up the PVE credential	7
Installing the package	8
Where the configuration lives	8
Where the configuration is <i>not</i>	8
Running the timer on exactly one host	8
First steps after installing	9
Where the numbers come from, and how a transport loses them	9
The six counters	9
Two things deliberately not used	10
The failure mode: a transport that cannot carry strings	10
Telegraf with a Prometheus remote-write output	10
What catches it	10
A reference implementation that is well tested	11
Other backends	11
What to do next	11
Configuration reference	12
schema_version	12
proxmox — the cluster API connection	12
proxmox.host	12
proxmox.port	12
proxmox.verify_ssl	12
proxmox.ca_file	12
proxmox.auth.username	12
proxmox.auth.password	13
proxmox.auth.token_id	13
proxmox.auth.token_secret	13
proxmox.ticket_refresh_seconds	13
proxmox.read_workers	13
prometheus — the metrics source	13
prometheus.url	13
prometheus.timeout_seconds	13
prometheus.username / prometheus.password	13
prometheus.bearer_token	14
metrics — Telegraf/InfluxDB name mapping	14
metrics.read_ops	14
metrics.write_ops	14
metrics.read_bytes	14
metrics.write_bytes	14
metrics.read_time_ns	14
metrics.write_time_ns	14
metrics.labels.vmid	14
metrics.labels.device	15
metrics.labels.node	15
metrics.extra_selector	15
metrics.rate_window	15

metrics.step	15
metrics.pvestatd_push_interval	16
window — the decision window	16
window.lookback	16
window.quantile	16
window.upper_quantile	16
window.min_coverage	16
load_weights — combining read/write and time/ops/bytes	17
load_weights.iotime	17
load_weights.ops	17
load_weights.bytes	17
load_weights.read_factor	17
load_weights.write_factor	17
groups — storage groups	17
groups[].name	17
groups[].storages[].id	18
groups[].storages[].capability_weight	18
groups[].storages[].reserve_factor	18
groups[].storages[].free_space.soft / groups[].storages[].free_space.hard	18
groups[].storages[].saturation_load	18
snapshot_reserve — the free-space floor	18
snapshot_reserve.factor	18
snapshot_reserve.min_free_bytes	19
snapshot_reserve.count_foreign_volumes	19
free_space — keep N bytes (or N%) free on top of the snapshot reserve	19
free_space.soft	19
free_space.hard	19
gates — deciding whether to act at all	20
gates.drift_threshold	20
gates.imbalance_threshold	20
gates.capacity_spread_threshold	20
gates.cooldown_per_disk	20
gates.cooldown_per_storage	20
migration — cost, bandwidth and the payback rule	20
migration.bwlimit_bytes_per_sec	20
migration.source_load_weight	21
migration.target_load_weight	21
migration.payback_horizon	21
migration.payback_ratio	21
migration.max_single_move_duration	21
migration.account_saferemove_wipe	21
migration.wipe_load_weight	22
migration.saturation_ceiling	22
migration.assume_thick_provisioning	22
migration.tiny_disk_bytes	22
objective — the solver's trade-off weights	23
objective.spread_metric	23
objective.alpha_spread	23
objective.beta_move_count	23

objective.gamma_move_bytes_per_tib	23
objective.kappa_vm_affinity	24
objective.delta_capacity_spread	24
objective.affinity_counts_pinned_disks	24
objective.reserve_violation_penalty	24
solver — which backend plans	25
solver.backend	25
solver.time_limit_seconds	25
solver.mip_gap	25
solver.heuristic_iterations	25
execution — how (and whether) moves actually happen	25
execution.mode	25
execution.max_concurrent_migrations	26
execution.max_migrations_per_run	26
execution.max_concurrent_per_storage	26
execution.max_replans_per_run	26
execution.abort_on_failure	26
execution.poll_interval_seconds	27
execution.locks.wait_timeout	27
execution.locks.poll_interval	27
execution.locks.on_timeout	27
execution.locks.task_retry_limit	27
execution.locks.task_retry_backoff	27
execution.source_release.wait	27
execution.source_release.timeout	27
execution.time_windows[].days	27
execution.time_windows[].start / execution.time_windows[].end	28
exclude — what DRS never touches	28
exclude.vmsids	28
exclude.disks	28
exclude.storages	28
exclude.tags	28
exclude.skip_vms_with_snapshots	28
exclude.running_only	28
exclude.include_unused_disks	28
report	29
report.warn_pinned_load_fraction	29
state	29
state.path	29
forecast — history beyond the plain quantile	29
forecast.model	29
forecast.seasonal_lookback_days	29
forecast.holt_winters.seasonal_periods	29
forecast.holt_winters.trend	30
forecast.holt_winters.seasonal	30
forecast.holt_winters.residual_z	30
support — diagnostic bundles	30
support.salt_path	30
support.bundle_dir	30

support.max_series_points	30
support.capture_range	30
monitoring — telling your monitoring system what the last run did	31
monitoring.status_file	31
Verifying your setup	31
Running it	31
What each finding means	32
If it fails	32
Reading show-load and verify-storages	33
show-load	33
The Group <name> → ACT/NO ACTION line	34
verify-storages	36
A negative saferemove_throughput is normal	36
Diagnostic bundles: collect-testdata and --replay	37
collect-testdata: capturing a bundle	37
What is in a bundle, and what is not	38
--replay: running against a bundle offline	39
Sending one to the project	39
Reading plan	40
Reading a move line	41
The payback: line	42
What plan does not yet do	42
Reading apply	43
The [y]es/[n]o skip/[a]ll remaining/[q]uit prompt	44
Reading auto mode	44
Concurrent execution	45
What “done” means for one move, and why it can take a while	45
Failure and abort_on_failure	46
When something cannot be read	46
state.json: what a real run actually changes	47
Crash and two-instance recovery	47
--json	47
Reading explain	48
The objective: line	48
no moves made: ... / closest alternative:	49
measured load:	49
-v: the data source: line	49
pinned (not movable this run):	50
cannot fully consolidate:	50
pinned load ... ; best achievable spread given pins:	50
Safety properties, exit codes, and what this build actually does	51
What is safe, unconditionally	51
Exit codes	51
What this build actually implements	51

Optional dependencies	54
Logging: what lands where, and what an unattended run records	54
Verbosity	55
The decision trail	55
Unattended runs log this without being asked	55
Text or JSON	55
Under systemd	56
Monitoring: the status file	56
What is in the file	56
What makes a run OK, WARNING or CRITICAL	56
Freshness: why every run rewrites the file	57
Wiring it into Nagios/NRPE	57

What you need

- **Proxmox VE 9.2**, or a management host with network access to a PVE 9.x cluster's API (tcp/8006) — the tool does not need to run on a cluster member.
- **A Prometheus-compatible backend** already receiving PVE's per-disk blockstat metrics via the InfluxDB external metric server — plain Prometheus, VictoriaMetrics, or gigapipe on ClickHouse, which is what this project is dogfooded against. If your cluster's dashboards already show per-VM disk I/O, the export side is already on. That is not the same as all six counters having survived the trip, which is a real and silent failure mode: read [05-metrics-pipeline.md](#) before writing the metrics block, and run `pve-storage-drs verify-metrics` before relying on any plan. No new exporter or collector is required.
- **A PVE user or API token** with read access to VM and storage inventory and permission to call `move_disk`. An API token is preferred for unattended operation (`execution.mode: auto`); a user-name/password pair works for interactive use. See "Setting up the PVE credential" below for the exact privileges — the obvious-looking "just grant Audit everywhere" is not enough, and the gap is silent rather than an error.

Setting up the PVE credential

Grant, on **every storage the tool will manage** (the shared, image-holding storages in your groups, not backup targets):

- **Datastore.Audit and Datastore.Allocate** — not Audit alone. `GET /nodes/{node}/storage/{storage}/content` which the tool uses to cross-check each disk's real allocated size, silently returns an empty list under Audit-only permission: a clean 200 response with no content and no error, indistinguishable from "this storage genuinely has nothing on it" unless you already know to be suspicious. Audit alone is enough for every *other* read call the tool makes; this one endpoint is the exception, verified against a live PVE 9.2.11 cluster (see [IMPLEMENTATION_PLAN.md](#) section 3.5's note). Getting this wrong does not make the tool fail loudly — it makes it under-count what already occupies each storage, which quietly erodes the **snapshot reserve**: the free capacity `pve-storage-drs` always keeps clear on every managed storage (sized from `snapshot_reserve.factor/free_space.soft` — see [10-configuration.md](#)) so that a snapshot, or a disk mid-migration existing on two storages at once, never runs a LUN out of space. An under-counted storage looks to have more headroom than it really does, so the tool plans and executes migrations that erode a safety margin it believes it is still protecting.
- Grant it **at each storage's own path** (`/storage/<id>`), not only at the parent `/storage` — that is the grant confirmed to work.
- Also add `VM.Audit` cluster-wide (`/`) for VM inventory and config, which `PVEAuditor` already includes if you use that built-in role as a base.

If you are using an API token (recommended — see above), Proxmox's privilege separation means **the token has its own, separate ACL entries from its owning user**. A role granted to the user alone has no effect on the token, and a role granted to the token alone has no effect either unless the user has it too: the token's effective permission is the *intersection* of the two. Grant the same roles to both the user and `user@realm!tokenId`, or disable privilege separation on the token (Datacenter → Permissions → API Tokens → uncheck "Privilege Separation") so it always matches its user's permissions exactly.

A concrete, minimal setup: create a role (e.g. `pve-storage-drs`) with `Datastore.Allocate`, `Datastore.Audit`, `VM.Audit`, then add it as an ACL entry for both the user and the token at `/` for `VM.Audit`, and at each managed storage's `/storage/<id>` path for the datastore privileges.

Two further, VM-level privileges are needed once apply executes a move — `VM.Config.Disk` and `VM.Migrate`, both cluster-wide (`/`) or at least on every VM whose disks live in a managed group, granted

to both the user and the token exactly as above. Not needed for `verify-metrics`, `show-load` or `verify-storages`, which only read; apply needs them for `move_disk` itself, in every `execution.mode` (`30-safety-and-status.md`). Grant them now alongside the privileges above so the credential does not need revisiting later; if you set up the credential read-only for now, add `VM.Config.Disk`, `VM.Migrate` to the role before you first run `apply`.

Installing the package

On a Debian trixie host (which is what Proxmox VE 9.x is built on):

```
apt install pve-storage-drs
```

This installs the `pve-storage-drs` executable, its manpage, the example configuration at `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml`, and the whole documentation set under `/usr/share/doc/pve-storage-drs/`: this manual as `manual/*.md`, the internals guide as `internals/*.md` and the specification as `IMPLEMENTATION_PLAN.md`, each also as a typeset PDF. The Markdown is deliberately left uncompressed — a cluster node has no GUI and often no way to get a PDF off it, so less `/usr/share/doc/pve-storage-drs/manual/20-verifying-metrics.md` has to work on the machine that has the problem.

`coinor-cbc` and `python3-pulp` are Depends: `apt install pve-storage-drs` always gets a real MILP solver, no separate step needed. The tool still plans without one — using the dependency-free heuristic of `IMPLEMENTATION_PLAN.md` section 5.5 — but that is no longer what a Debian install lands on by default; it is what a non-Debian `pip install` with no solver extra, or a solve that fails or times out, falls back to.

Where the configuration lives

`/etc/pve/drs.yaml`. On a cluster member this path is on `pmxcfs`, the cluster filesystem, so the file is automatically the same on every node — there is exactly one copy and no question of which one is authoritative. A management host that is not itself a cluster member simply keeps an ordinary file at that same path.

Two things follow from that placement, and both matter before you put a password in the file:

- **An edit is live on every node the instant it is saved.** There is no staging step. This is why configuration mistakes are always hard failures (see [30-safety-and-status.md](#)) rather than warnings, and why `dry-run` is the default execution mode.
- **Files under `/etc/pve` are group-readable by `www-data`** — the PVE web server’s user. Check `ls -l /etc/pve/drs.yaml` on your cluster before deciding whether a plaintext password: is acceptable; the safer choice is an API token with `PVE_TOKEN_SECRET` set in the environment (or a `systemd` credential) rather than written into the file at all.

Copy `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml` to `/etc/pve/drs.yaml` and edit it — every key is commented with its default and what it does. The full reference is [10-configuration.md](#).

Where the configuration is *not*

`state.path` (default `/var/lib/pve-storage-drs/state.json`) is deliberately **not** on `/etc/pve`. It is rewritten on every run, `pmxcfs` writes need cluster quorum, and the state it holds — cooldown timestamps, the load vector at the last balance, the run lock — is meaningful only to the one host that is actually running the timer. See [10-configuration.md#statepath](#).

Running the timer on exactly one host

Putting the configuration on `pmxcfs` does not make the tool cluster-aware. `state.json` is node-local: each node would keep its own cooldowns, its own drift baseline and its own run lock, which do not coordinate

with each other. **Enable the systemd timer on exactly one host.** The one thing that *does* cross the cluster is a startup scan for in-flight move_disk UPIDs owned by the DRS user, which is what keeps two accidental instances from actively conflicting (they degrade to “slow and redundant” instead).

First steps after installing

1. Read [05-metrics-pipeline.md](#) — where the six per-disk counters come from, and how a Telegraf-to-Prometheus transport can drop one of them without ever logging an error.
2. Read [10-configuration.md](#) and adjust the storage groups, thresholds and weights for your cluster.
3. `pve-storage-drs -c /etc/pve/drs.yaml verify-metrics` — confirms the configured metric and label names actually exist in your backend, and cross-checks the six metrics against each other. See [20-verifying-metrics.md](#).
4. See [30-safety-and-status.md](#) for exactly which commands this build supports end to end today.

Where the numbers come from, and how a transport loses them

`pve-storage-drs` collects nothing itself. It ships no exporter, installs no agent on your nodes and opens no connection to a VM. Every number in every plan comes out of a Prometheus-compatible backend you already run, via a pipeline PVE already has.

Read this page before you write the metrics block in [10-configuration.md](#). The pipeline is where this tool’s inputs get quietly damaged, and a damaged input produces a confident plan rather than an error.

The six counters

Per VM disk, per push:

<code>rd_operations</code>	<code>wr_operations</code>	how many I/Os
<code>rd_bytes</code>	<code>wr_bytes</code>	how much data
<code>rd_total_time_ns</code>	<code>wr_total_time_ns</code>	how long the device was busy

They are QEMU’s own `query-blockstats` figures. `pvestatd` already gathers them on every status push — it calls `vmstatus(undef, 1)`, and the `full` flag is what makes it issue the QMP query — and PVE’s **InfluxDB external metric server** already exports the whole stats hash without filtering.

PVE/Status/InfluxDB.pm flattens that nested hash so the first level becomes the measurement (`blockstat`), the second becomes a **tag** (`instance=scsi0`), and the leaves become fields. A series therefore arrives looking roughly like:

```
blockstat_rd_operations{vmid="101", instance="scsi0", nodename="pve01", host="db-01"}
```

The device id being a **label** rather than part of the metric name is the whole reason this approach works: it is what makes `per-disk rate()` and aggregation across drives possible at all.

Note the label names in that example. PVE’s own tag for the drive id is `instance`, which **collides with Prometheus’s own scrape-target label of the same name**, and many Telegraf configurations rename or overwrite one of the two. That is why the tool never hardcodes them: `metrics.labels.vmid`, `.device` and `.node` are configuration keys naming whatever labels your deployment actually produces, defaulting to `vmid`, `instance` and `nodename`. `verify-metrics` warns when `.device` is still `instance` — not because that is wrong, but because it is the one name where you have to confirm which of the two survived before anything joins on it.

So the data exists cluster-wide before this tool is installed, and there is nothing new to deploy. [IMPLEMENTATION_PLAN.md §3.1](#) carries the source excerpts if you want to confirm any of the above against `pve-manager` and `qemu-server` yourself.

Two things deliberately not used

- **RRD.** GET /nodes/{node}/qemu/{vmid}/rrddata exposes only VM-aggregate diskread/diskwrite in bytes per second — no per-disk breakdown, no operation counts, no latency. It cannot answer the question this tool asks.
- **The OpenTelemetry metric server** (new in PVE 9.1), despite being the more modern-looking option. It appends a nested key to the metric *name* rather than carrying it as an attribute, so `blockstat.scsi0.rd_operations` becomes `proxmox_vm_blockstat_scsi0_rd_operations_total`. The device is baked into the name, which makes aggregation across devices impossible without `label_replace` gymnastics and makes metric-name cardinality unbounded. The InfluxDB path is strictly better here.

The failure mode: a transport that cannot carry strings

This is the most common way to get a wrong answer out of `pve-storage-drs`, and it never announces itself.

PVE’s InfluxDB output speaks **InfluxDB line protocol**, which carries **string** fields alongside numeric ones. Two properties of that protocol combine badly with a Prometheus-shaped destination:

1. **A field’s type is fixed by its first write.** Once a given series’ field has arrived as a string, it is a string for that series from then on — permanently, not until the next push corrects it.
2. **Prometheus and OpenMetrics have no string sample type.** A sink that cannot represent a string field does the only thing available to it: it drops the field.

Put together: one non-numeric value, once, possibly months ago, for one disk, is enough to make one of the six counters vanish for that series while the other five keep working perfectly. Nothing logs an error. Your dashboards keep drawing, because they rarely plot all six.

What `pve-storage-drs` then sees is a disk that does no writes, or a disk whose device was never busy. The load model has no way to tell that apart from a genuinely idle disk — the data does not say “missing”, it says nothing at all — so it plans around a number that is simply false.

Telegraf with a Prometheus remote-write output

This is the prominent example, and it is prominent because it is what most people build first:

```
inputs.influxdb_listener → (no processors) → outputs.prometheus_remote_write
```

`outputs.prometheus_client` and any other OpenMetrics-shaped output behave the same way, as does any other bridge from PVE’s InfluxDB output into a Prometheus data model. **Unless the pipeline is configured deliberately** — converting the string fields, or dropping them explicitly with a processor so you know what you lost — it will silently discard fields, and you will find out from a plan that does not make sense rather than from a log line.

This is not an argument against Telegraf. It is an argument for configuring its processors on purpose, and for running `verify-metrics` afterwards rather than assuming.

What catches it

`verify-metrics` cross-checks the full (`vmid`, `device`) set that each of the six metrics reports against the other five, and warns — naming the metric and the disk — whenever one metric’s set is a strict subset of another’s. This costs your Prometheus nothing extra: it reuses the six instant queries that step of `verify-metrics` was already making in order to print a sample series.

Treat that warning as a real defect in the pipeline, not as noise. Fixing the pipeline does not retroactively restore the samples you never received, and by point 1 above it does not necessarily restore future ones

either: as long as the affected series keeps arriving under the same identity with the field pinned to a string type, it stays dropped. Re-check with `verify-metrics` after any change rather than assuming.

A reference implementation that is well tested

gigapipe with ClickHouse as the database, fed directly from PVE's InfluxDB output.

gigapipe accepts **the InfluxDB protocol exactly as PVE exports it** on the ingest side, and serves the Prometheus HTTP query API — `/api/v1/query`, `/api/v1/query_range`, `/api/v1/label/<name>/values`, which is the entire surface this tool uses — on the read side. So PVE's own metric server writes to it directly and pve-storage-drs reads from it directly: there is no Telegraf, no output plugin and no format bridge anywhere in the path, and so nothing that can drop a field on type grounds.

It is the backend this project is dogfooded against. Both bundles in `tests/corpus/` were captured from a cluster running it and record it as their `prometheus_backend`, so the regression suite runs against real gigapipe-shaped data on every `make check`.

To use it, point `prometheus.url` at gigapipe's query endpoint. Nothing else in this manual changes — including at the `metrics.step/metrics.rate_window` default: a recently-updated gigapipe was found, live, to return zero series for a range query whose `step` reaches its `rate()` window's duration (exactly that default), and this tool works around it unconditionally on every backend rather than asking you to change your configuration for gigapipe's sake. See `metrics.step` in `docs/manual/10-configuration.md` for what the workaround changes about what actually gets fetched.

gigapipe also ingests OpenTelemetry, but **that path is untested here** and is not the way to use it with this tool. Feeding it from PVE's OpenTelemetry metric server instead of the InfluxDB one would reintroduce the device-in-the-metric-name problem described above, whatever the backend does with it afterwards.

Other backends

Plain **Prometheus** and **VictoriaMetrics** both work. Every query this tool issues — `rate()`, `quantile_over_time`, subquery ranges, and `/api/v1/label/___name___/values` — is supported by all three backends, and `prometheus.url` is the only setting that distinguishes them.

Whatever you use, the pipeline in front of it is still yours to get right, and `verify-metrics` is still the thing that tells you whether you did.

What to do next

1. Work out which transport your cluster actually uses today. The PVE side is the external metric server — `/etc/pve/status.cfg`, which is what Datacenter → Metric Server writes; if there is no InfluxDB entry there, nothing described on this page is happening yet and that is the first thing to set up.

Do **not** take an existing “VM disk I/O” dashboard as evidence that it is. Such a dashboard may be drawing PVE's RRD data, or series from `prometheus-pve-exporter`, neither of which carries per-disk blockstat at all. Query your backend for `blockstat_rd_operations` directly and look at whether the result carries a per-drive label.

2. Set the `metrics` block in `/etc/pve/drs.yaml` to match the names that transport produces — they are configuration, not constants, because Telegraf's measurement/field join character and tag renaming vary per deployment. `10-configuration.md` documents every key with its default.
3. Run `verify-metrics` and read every finding.

Configuration reference

Every key pve-storage-drs reads is documented here: type, unit, default, what it interacts with, and what happens if you set it too high or too low. A key that exists in the schema but not here, or here but not in the schema, is a bug — `tests/unit/test_documentation.py` asserts the two never drift apart.

`config/drs.example.yaml` in the source tree (installed to `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml`) carries the same defaults with inline comments; this page is the full explanation each comment points back to.

`schema_version`

Integer, required, no default.

The config format's version. pve-storage-drs refuses to start on a `schema_version` whose **major** number it does not understand, naming the version it does understand in the error, rather than guessing at fields from a future incompatible format. Currently 1.

proxmox — the cluster API connection

`proxmox.host`

String, required, no default.

Any cluster member's hostname or IP. The API is reached at `https://<host>:<port>/api2/json`.

`proxmox.port`

Integer, default 8006.

The PVE API port. Change this only if you have put something unusual (a reverse proxy) in front of the API.

`proxmox.verify_ssl`

Boolean, default true.

Whether to verify the API's TLS certificate. Set false only for a self-signed certificate you cannot otherwise trust, and prefer `proxmox.ca_file` instead where possible — disabling verification also disables protection against a compromised network path to every node's API.

`proxmox.ca_file`

String path or null, default null.

A CA bundle to verify the API's certificate against, when it is not signed by a certificate already trusted by the host running pve-storage-drs.

`proxmox.auth.username`

String or null.

A PVE user, e.g. `drs@pve`. Required unless `proxmox.auth.token_id` is set. Username/password authentication is what the original requirement asked for; an API token (below) is preferred for `execution.mode: auto` because it needs no interactive ticket refresh and can be scoped tightly: the tool's actual minimum privilege is `Datastore.Audit` **and** `Datastore.Allocate` on every storage in the config's groups, plus `VM.Config.Disk` and `VM.Migrate` (or an equivalent custom role) on the VMs it may move. `Datastore.Audit` alone is not enough — PVE's per-volume content listing silently returns an *empty* result with no error when only Audit is granted, which would make every tracked disk look untracked rather than raising anything you'd notice. Grant the role per storage rather than relying on propagation from the parent `/storage` path, and remember that an API token's effective permission is the **intersection** of the user's own grants and the token's own: both need the grant, or neither has it. (IMPLEMENTATION_PLAN.md section 3.5.)

proxmox.auth.password

String or null, default null, environment: PVE_PASSWORD.

Leave this null in the file and set PVE_PASSWORD in the environment (or a systemd credential) instead, especially when the config lives on /etc/pve and is therefore group-readable by www-data. A password given in the file is used as-is and is **not** overridden by the environment variable — see [00-installation.md](#) for why the file's own permissions matter here.

proxmox.auth.token_id

String or null, default null.

An API token id, e.g. drs@pve!balancer. Set this instead of username/password for unattended operation; a token needs no ticket refresh loop and can be revoked independently of the user's password.

proxmox.auth.token_secret

String or null, default null, environment: PVE_TOKEN_SECRET.

The secret half of the API token, with the same environment-variable preference as proxmox.auth.password.

proxmox.ticket_refresh_seconds

Duration, default 3000 (50 minutes).

How often a username/password ticket is refreshed. PVE tickets last approximately two hours; refreshing well before that avoids a mid-run authentication failure. Not used for API-token authentication, which needs no ticket. Implemented by overriding proxmoxer's own (otherwise fixed-at-3600s) refresh interval after login, since proxmoxer has no constructor option for it; if a ticket somehow still gets rejected mid-run regardless (a long confirm-mode wait, a suspended process, a clock jump — see [50-pve-api.md](#)), the client transparently logs in again from scratch and retries the one call that failed, once, before giving up.

proxmox.read_workers

Integer, default 12.

Size of the bounded thread pool used to fetch per-VM configuration — GET /nodes/{node}/qemu/{vmid}/config has no batch form, so this is what keeps a several-hundred-VM cluster's topology read from being serial. Raising it trades API server load for wall-clock time; 8–16 is the range the plan suggests. Too high a value on a small PVE API server can itself become the bottleneck. Only the per-VM fetch is parallelized; the cluster-wide and per-storage calls ([50-pve-api.md](#)) still run once each, sequentially, before it starts.

prometheus — the metrics source**prometheus.url**

String, required, no default.

The base URL of your Prometheus, e.g. <http://prometheus.example.com:9090>.

prometheus.timeout_seconds

Duration, default 30.

Per-request timeout against Prometheus. Too low a value on a busy or long-range query makes verify-metrics and the load model fail spuriously; too high a value delays noticing that Prometheus is actually unreachable.

prometheus.username / prometheus.password

String or null, default null.

HTTP basic auth credentials, if your Prometheus requires them. Ignored when `prometheus.bearer_token` is set.

`prometheus.bearer_token`

String or `null`, default `null`.

A bearer token for Prometheus instances behind an auth proxy that expects one. Takes precedence over basic auth when both are set.

metrics — Telegraf/InfluxDB name mapping

Telegraf's naming is deployment-specific (`IMPLEMENTATION_PLAN.md` section 3.3), so every name below is configuration, never a constant. Run `pve-storage-drs verify-metrics` after changing any of them — see [20-verifying-metrics.md](#).

`metrics.read_ops`

String, default `blockstat_rd_operations`.

The Prometheus metric name for per-disk read operations/second (raw counter, before `rate()`).

`metrics.write_ops`

String, default `blockstat_wr_operations`.

As `metrics.read_ops`, for writes.

`metrics.read_bytes`

String, default `blockstat_rd_bytes`.

The Prometheus metric name for per-disk bytes read (raw counter).

`metrics.write_bytes`

String, default `blockstat_wr_bytes`.

As `metrics.read_bytes`, for writes.

`metrics.read_time_ns`

String, default `blockstat_rd_total_time_ns`.

The Prometheus metric name for cumulative read I/O time in nanoseconds. This is the **primary** load signal by default (`load_weights.iotime`): by Little's law, $\text{rate}(\text{rd_total_time_ns} + \text{wr_total_time_ns}) / 1\text{e}9$ is the average number of I/O requests in flight on that disk — dimensionless, additive across disks on the same storage, and directly comparable between storages of different size and speed. It also *self-weights* reads against writes automatically: an operation that takes eight times as long to service (a large sequential write against a small random read, say) counts eight times as much, with no manual read/write tuning needed. Pure IOPS treats every operation as equal and so under-counts large sequential load; pure throughput does the reverse and under-counts small random load. `ops` and `bytes` (see `load_weights` below) remain available as additional weighted terms for a policy the array's own timings do not capture. (`IMPLEMENTATION_PLAN.md` section 4.)

`metrics.write_time_ns`

String, default `blockstat_wr_total_time_ns`.

As `metrics.read_time_ns`, for writes.

`metrics.labels.vmid`

String, default `vmid`.

The Prometheus label carrying the VM id.

metrics.labels.device

String, default instance.

The Prometheus label carrying the drive id (`scsi0`, `virtio0`, ...). **Confirm this with `verify-metrics`.** PVE's own tag of this name collides with Prometheus's own `scrape-target` instance label, and many Telegraf configurations rename or overwrite it; `verify-metrics` warns loudly when this is still set to the literal string `instance`.

metrics.labels.node

String, default nodename.

The Prometheus label carrying the PVE node name — what the auto-derived filter below is built from: the tool fetches this cluster's own node list from the PVE API and scopes every query to `<this label>=~"<node1>|<node2>|..."`, so a Prometheus shared by more than one PVE cluster (or by anything else emitting a same-named metric) cannot silently sum in a same-numbered vmid from somewhere else. This is the default filter for every run; `verify-metrics` never applies it — see `metrics.extra_selector` below.

Confirm what's actually there with `verify-metrics` before assuming the default is right for your Prometheus: it reports each configured metric's sample series labels so you can check this key names a real one.

metrics.extra_selector

String or null, default null (auto).

Overrides the automatic node-scoping filter above with a raw PromQL label matcher, inserted verbatim into every query's vector selector (e.g. `{metrics.extra_selector}` becomes part of `rate(blockstat_rd_operations{cluster="mycluster"}[5m])`). Set this when the node-list filter isn't quite what you need — your own Telegraf/InfluxDB tagging scheme doesn't put the exact PVE node name in `metrics.labels.node`, you want to scope by something else (a cluster-naming tag your deployment happens to carry, say), or the restriction needed is something else entirely. Write exactly what your label scheme needs, e.g. `nodename=~"pve01|pve02|pve03"` or `cluster="mycluster"`; the value is used as-is, with no further quoting or escaping. `pve-storage-drs verify-metrics` applies this override too when set, but never derives its own filter automatically — it is deliberately independent of the PVE API entirely.

There is deliberately no “cluster label” auto-scoping tier above the node-list default: an earlier revision of this tool assumed such a label was standard practice across this project's deployments, which turned out to be wrong — nothing in the Telegraf/InfluxDB pipeline guarantees one. Use `metrics.extra_selector` for that instead, if your Prometheus actually carries such a label.

metrics.rate_window

Duration, default 5m.

The inner range passed to `rate()`. Must be at least four times `metrics.pvestatd_push_interval`, or `rate()` sees too few points to be meaningful — `verify-metrics` measures the *observed* sample spacing of a live series and errors if this rule is violated against reality, not just against the declared `pvestatd_push_interval`.

metrics.step

Duration, default 5m.

The sampling step used for range queries (forecasting, coverage checks). Independent of `metrics.rate_window`, though the two are usually set equal — which is also the one setting a workaround below silently changes what actually gets fetched at.

Whenever `metrics.step` $\geq$ `metrics.rate_window` (true at the usual-equal default), the collector does not send `metrics.step` to Prometheus as-is: a live-confirmed gigapipe bug returns zero series for any `rate()`-based range query or `quantile_over_time` subquery once the query’s own step reaches the function’s range-vector duration, so this tool queries at half that step (or the largest whole-second step below it that divides `metrics.step` evenly, at a wider ratio) and reassembles the configured grid from the result. Every disk’s coverage, load history and forecaster input comes back numerically identical to what a plain `metrics.step` query would have returned on an unaffected backend — except the `quantile_over_time` decision statistic itself, whose inner evaluation runs on the denser grid unconditionally, on every backend (a small, real shift in the reduced 95th-percentile value; see `docs/internals/30-metrics.md`). The *stored* sample count this produces — what `collect-testdata --estimate` prints and `support.max_series_points` compares against — is correspondingly doubled (or more) at the same ratio.

`metrics.pvestatd_push_interval`

Duration, default 60s.

How often PVE’s `pvestatd` pushes metrics — a PVE-side setting this tool cannot read from the API, so it must be declared here. `verify-metrics` cross-checks it against the *observed* sample spacing of a live series and errors if they disagree by more than 20%, which is what stops a stale declaration from silently invalidating the `rate_window` rule above.

window — the decision window

This is the period whose load is balanced. It is **not** the amount of history a forecaster needs to fit — see `forecast.model` below and `IMPLEMENTATION_PLAN.md` section 10.1.

`window.lookback`

Duration, default 24h.

The trailing window each disk’s load is reduced over. Must be at least as long as the configured `forecast.model` needs (`forecast.model: holt_winters` at its default `seasonal_periods` needs 48h, which a 24h lookback can never supply — this is rejected at startup, not silently degraded).

`window.quantile`

Fraction in (0, 1), default 0.95.

The point-estimate quantile: each disk’s load is the p95 of its raw signal over `window.lookback`, not the mean, so one traffic spike neither triggers nor suppresses a migration.

`window.upper_quantile`

Fraction in (0, 1), default 0.99.

The quantile the **saturation guard** actually consumes — must be $\geq$ `window.quantile`. (The saturation guard is the migration-time safety check, `migration.saturation_ceiling` below, that defers a move if it would push a storage’s forecasted load past a configured ceiling; it runs only for a storage that sets `groups[].storages[].saturation_load`.) Being wrong in the direction of “busier than it looks” costs the guard deferring a move that was actually safe; the other direction risks the guard missing a mirror that pushes a storage past saturation. The optimizer itself still decides placement from `window.quantile`, the point estimate, not this upper bound; wiring the upper bound into the optimizer’s own input remains future work. (`IMPLEMENTATION_PLAN.md` §10.1’s “As built” note.)

`window.min_coverage`

Fraction in (0, 1], default 0.80.

A disk whose sample coverage over `window.lookback` falls below this fraction has its data rejected for this run; its last known load from `state.json` is used instead, flagged in the report. Never treated as zero —

that would silently invite migrations *onto* a busy but under-sampled storage.

load_weights — combining read/write and time/ops/bytes

Each disk's load is a single scalar, in **average in-flight I/O requests**, blended from three raw per-disk signals — I/O time, operations/second and bytes/second — after applying the read/write asymmetry factors below. The three raw signals have wildly different magnitudes (in-flight I/O is roughly 0–10, ops/s can run into the tens of thousands, bytes/s into the billions), so they cannot be weighted directly: each is first normalized against its own group's total (which makes the three comparable, but throws away their physical meaning), the normalized values are blended using the weights below, and the blend is then rescaled back onto the in-flight-I/O scale using the group's total I/O time. Under the defaults (`iotime: 1.0`, `ops: 0.0`, `bytes: 0.0`) that rescale is an exact identity, so a disk's load is simply its own I/O-time term — the normalize/blend/rescale machinery only changes the result once ops or bytes carries a non-zero weight. The absolute scale this produces matters beyond the blend itself: `migration.source_load_weight` and its siblings below assume that 1.0 in-flight request at a mirror endpoint is directly comparable to a disk's own load, which is only true because of this rescale. (Derived in full, with the normalization formula, in `IMPLEMENTATION_PLAN.md` section 4.)

load_weights.iotime

Weight, default 1.0.

Weight on the I/O-time term ($\text{rate}(\text{rd_total_time_ns} + \text{wr_total_time_ns}) / 1\text{e}9$), the primary signal by default: it is Little's-law in-flight I/O, additive across disks and self-weighting between large sequential and small random access with no manual read/write tuning needed.

load_weights.ops

Weight, default 0.0.

Weight on the operations/second term. Non-zero only if you want IOPS to influence placement independently of I/O time.

load_weights.bytes

Weight, default 0.0.

Weight on the bytes/second term, for the same reason as `load_weights.ops`.

load_weights.read_factor

Weight, default 1.0.

Read/write asymmetry applied to the ops/bytes terms only (see `load_weights.write_factor`). Applying it to the `iotime` term as well double-counts, since I/O time already reflects the real read/write cost difference — do that only deliberately.

load_weights.write_factor

Weight, default 1.0.

As `load_weights.read_factor`, for the write side. A write-heavy RAID-6 array, for example, might set this above 1.0 on the ops/bytes terms to express the real cost asymmetry those terms cannot otherwise see.

groups — storage groups

groups[].name

String, required.

A group's name. A disk may only ever move between storages in its own group; there is no cross-group balancing (`IMPLEMENTATION_PLAN.md` section 5). Must be unique.

groups[].storages[].id

String, required.

A PVE storage id, or a /regex/ pattern — a value that both begins and ends with / — matching one or more storage ids (IMPLEMENTATION_PLAN.md section 11.4). A pattern is matched with `re.fullmatch` (case-sensitive) against the live cluster’s storage inventory on every run, so `/san-.*` picks up a LUN added after the config was written with no edit needed; its own entry’s `capability_weight/reserve_factor/saturation_load` apply to every storage it matches. A literal entry always overrides a pattern that also matches its storage, so one member of a pattern-matched family can still be pinned to different options. A storage may belong to **at most one** group (after pattern expansion) — belonging to two would make a disk’s eligible destinations ambiguous, and config/cluster validation rejects it, as does a pattern matching zero storages or two patterns in one group claiming the same storage.

groups[].storages[].capability_weight

Weight, > 0 , default 1.0.

The storage’s **relative** share of the group’s load — $u_s = L_s / c_s$ is what actually gets equalized, so a storage with half the spindles of its peers can be given 0.5 and will correctly be assigned half the load. Only the *ratios* within a group matter; the absolute value is meaningless and it is not a capacity.

groups[].storages[].reserve_factor

Weight or null, default null (inherits `snapshot_reserve.factor`).

Per-storage override of the group-wide snapshot reserve factor, for a storage whose snapshot behaviour genuinely differs from its peers.

groups[].storages[].free_space.soft / groups[].storages[].free_space.hard

Size, byte-unit string, percentage string ("`%`"), or null — each independently. null here means *inherit the global free_space.soft / .hard value below* (`$free_space` — the same per-storage-null-means -inherit rule `reserve_factor` above already has). One member of a /.../-matched family can still be pinned to a different requirement than its peers, the same way a literal entry already overrides a pattern’s `reserve_factor`.

A percentage is resolved against **this storage’s own capacity**, even when it comes from a pattern entry matching several differently-sized storages — "10%" demands different byte counts on a 20 TiB and a 2 TiB LUN, which is the point: “a tenth of the LUN free” is one policy applied per storage, not one number shared across the family.

groups[].storages[].saturation_load

Positive number or null, default null.

The storage’s approximate queue depth — the number of concurrent I/O requests it services before latency climbs super-linearly — in the same units as the load model (average in-flight I/O). Used only by the saturation guard on a migration’s mirror (see `window.upper_quantile` above). **Has no safe default:** an idle storage’s observed load is not its capacity, so leaving this null (the default) simply disables that one advisory check for the storage; the hard bounds always apply regardless — `migration.max_single_move_duration`, and the **transient reserve invariant**: the `snapshot_reserve` floor (below) checked against the storage’s actual state *while a migration is in flight*, when a moving disk’s source and target copies are both briefly fully allocated at once, not merely before and after. Obtain a real value for `saturation_load` from the array’s documented queue depth, or by observing where latency actually starts climbing.

snapshot_reserve — the free-space floor

snapshot_reserve.factor

Weight ≥ 0 , default 2.0.

Keep this many times the largest disk on a storage free at all times, including *during* a migration, not merely before and after — PVE 9’s volume-chain snapshots allocate a new full-size volume per snapshot, which is what this protects against. This is the constraint the tool never trades against balance (IMPLEMENTATION_PLAN.md section 5.3, (C5)).

snapshot_reserve.min_free_bytes

Size, default 0. **Deprecated syntax for free_space.soft below.**

An absolute floor, applied as $\text{reserve} = \max(\text{factor} * \text{largest_disk}, \text{min_free_bytes})$. Matters when a storage’s largest disk is small: with factor: 2.0 and a 10 GiB largest disk, the snapshot term alone would reserve only 20 GiB on a 20 TiB LUN — exactly what free_space.soft now expresses, with per-storage and percentage forms this scalar never had. Still accepted, and still works: if set alongside free_space.soft it is folded in as a floor on top of the resolved value (the larger of the two applies on every storage, never a “last one wins” substitution — a config warning names both keys when this happens), so upgrading is never a silent weakening. New configs should use free_space.soft instead.

snapshot_reserve.count_foreign_volumes

Boolean, default true.

Count volumes DRS does not manage (templates, ISOs, backups, other groups’ disks, orphans) against a storage’s used capacity. Strongly recommended: turning this off understates real usage and silently erodes the reserve it is meant to guarantee.

free_space — keep N bytes (or N%) free on top of the snapshot reserve

The Storage-DRS mandate (IMPLEMENTATION_PLAN.md section 5.3.1): an admin places a new VM on a storage this tool does not manage the free space of by snapshot behaviour alone, and the engine migrates data off it until the configured free space is free again — bypassing the drift/imbalance gates and the payback economics for exactly this reason, the same way a snapshot -reserve violation already does. soft and hard here are the *global* defaults; groups[].storages[].free_space.soft/.hard above override them per storage or per /.../-matched pattern, most specific wins.

free_space.soft

Size, byte-unit string, or percentage string (“N%”, $0 \leq N < 100$), default 0.

The **plan-endpoint** requirement: the number of bytes that must be free on a storage once the plan has fully run. $R_s = \max(\text{factor} * \text{largest_disk}, \text{soft})$ — the *larger* of the snapshot term and this floor wins, on every storage, always; neither term can erode the other. The default 0 changes nothing for a config that sets no knob in this block at all: the resulting model is identical to the pre-free_space snapshot-only floor. A storage that ends the plan below its soft requirement is in violation exactly like a snapshot-reserve breach, and the engine migrates disks off it until the requirement is met or reports the residual shortfall as unfixable (section 9.5) — it does not guarantee the requirement is *achievable*, only that it is pursued unconditionally.

free_space.hard

Size, byte-unit string, percentage string, or null, default null (= soft: no dip at all).

The **transient** floor: how far a storage may dip below soft while a disk is landing on it mid-migration (section 8.1) — a mirror target is fully allocated before its source releases anything, so a plan that lands a disk on a storage that is *heading toward* soft needs room to pass through it without ever crossing hard. The default null keeps the transient invariant exactly as strong as it was before free_space existed: no dip below soft at all. Set it strictly below soft only when you have deliberately decided to trade a bounded, planned dip for it — a transient dip can only ever land on a storage the *finished* plan leaves compliant, never used to permanently weaken the requirement. hard above soft is a config error: a floor stronger

than the requirement it is supposed to relax would make every plan for an already-compliant storage infeasible.

gates — deciding whether to act at all

`gates.drift_threshold`

Fraction, default 0.10.

Re-planning is skipped until the load vector has moved by this much in L1, relative to the vector recorded at the last executed balance. Lower reacts sooner and migrates more; below roughly 0.05 the tool will start chasing noise on a busy cluster. Interacts with `gates.imbalance_threshold`: drift decides *whether to look*, imbalance decides *whether to act*.

`gates.imbalance_threshold`

Fraction, default 0.20.

The minimum relative spread across a group's storages before a plan is actually built. A group under this threshold is left alone even if it has drifted. Also, unrelatedly, this same value doubles as the backtest error ceiling a non-quantile forecast.model (seasonal_naive/holt_winters) must stay within before its forecast is trusted to drive the saturation guard for that group's run — see `forecast.model` below for how that backtest works.

`gates.capacity_spread_threshold`

Fraction or null, default 0.25.

The data-spread counterpart of `gates.imbalance_threshold`: the minimum relative spread in fill fraction (bytes used / capacity) across a group's storages before a plan is built, evaluated even when the group's I/O is already perfectly balanced – in that case `objective.delta_capacity_spread` is what does the spreading. It can legitimately exceed 1 (a group filled to 5% that keeps 60% of its bytes on one storage deviates as much as a full one does). Unlike the drift/imbalance gates, this one **bypasses** them when it fires – a stable, balanced workload is not a reason to keep data concentrated on one storage. null disables the gate outright; the gate also cannot fire on a group with no data at all (mean fill 0).

`gates.cooldown_per_disk`

Duration, default 24h.

A disk that moved within this long is pinned in place for planning purposes. Prevents a disk from being shuffled back and forth across two still-noisy storages.

`gates.cooldown_per_storage`

Duration, default 1h.

A storage involved in a migration within this long accepts no new incoming moves. **Must exceed the implied wipe time of that storage's largest disk** (`largest_disk / |saferemove_throughput|` — the magnitude, see [verify-storages](#) on why that value is often negative) or the next run will plan onto a storage that is still draining and stall — `pve-storage-drs verify-storages` computes this and warns when the configured cooldown is too short.

migration — cost, bandwidth and the payback rule

`migration.bwlimit_bytes_per_sec`

Size/s, default 209715200 (200 MiB/s).

Passed to `move_disk` as `bwlimit` (converted to KiB/s at the API call site — the API's own unit, never bytes/s). Also the divisor in the mirror-duration estimate the payback test uses: a move's mirror is assumed to

take $\text{disk_bytes} / \text{bwlimit_bytes_per_sec}$ seconds (see `migration.payback_ratio` below for the full cost/benefit comparison).

`migration.source_load_weight`

Weight, default 1.0.

Additional in-flight I/O charged to the **source** storage while a mirror is running — a mirror is one sequential reader, so 1.0 is the natural value in the same units as the load model.

`migration.target_load_weight`

Weight, default 1.0.

As `migration.source_load_weight`, for the target (one sequential writer).

`migration.payback_horizon`

Duration, default 365d.

The horizon H over which a plan's imbalance, data-spread and VM-affinity reduction are assumed to persist — $\text{benefit} = (\alpha_{\text{spread}} * \Delta E + \delta_{\text{capacity_spread}} * \Delta F + \kappa_{\text{vm_affinity}} * \Delta A) * H$, where each Δ is the improvement the plan buys in that term (its value before the plan minus its value after: ΔE for imbalance, ΔF for data spread, ΔA for VM-affinity fragmentation — see objective below for what each term measures). ΔA may be negative (a balance move that splits a VM pays for that fragmentation out of its other gains — `IMPLEMENTATION_PLAN.md` section 7.2). Setting this to 0 disables the payback test entirely (`IMPLEMENTATION_PLAN.md` section 11.1 rejects that at config-load time: `payback_horizon > 0` is required). This is an explicit assumption about how long a placement lasts, not about operator patience: a migration's cost is paid once and early, while its benefit accrues for as long as the workload keeps running on the new placement — infrastructure timescales are months to years, and the default assumes at least one year. Configuration validation warns (does not error) below 30d, where the test starts rejecting real, slow-accruing benefit again; lower it toward the actual lifetime of your VMs only for genuinely short-lived fleets (CI runners, render farms, lab clusters).

`migration.payback_ratio`

Weight > 0 , default 10.0.

λ : a plan is only accepted when $\text{benefit} \geq \text{payback_ratio} * \text{cost}$, summed over every disk the plan moves. Both sides are in **load-seconds** — average in-flight I/O requests (the unit `load_weights` produces, see `metrics.read_time_ns` above) multiplied by seconds — which is what makes the comparison meaningful. A disk's cost (`cost_d`) is the extra in-flight I/O the migration itself imposes: `migration.source_load_weight` on the source and `migration.target_load_weight` on the target for the mirror, which takes $\text{disk_bytes} / \text{migration.bwlimit_bytes_per_sec}$ seconds, plus — when `migration.account_saferemove_wipe` is on and the source storage has `saferemove` set — `migration.wipe_load_weight` on the source for as long as the old volume takes to be zeroed. Benefit is the improvement the plan buys in imbalance, data spread and VM affinity, each weighted exactly as in the objective section below and held for `migration.payback_horizon` (see above for that formula). This is the numeric form of “migrating a very large disk might generate more traffic than it saves.” (`IMPLEMENTATION_PLAN.md` section 7.2.)

`migration.max_single_move_duration`

Duration, default 6h.

A hard per-move ceiling on `duration_mirror + duration_wipe`. A move predicted to take longer than this is rejected outright regardless of the aggregate payback test.

`migration.account_saferemove_wipe`

Boolean, default true.

Include the post-move saferemove zeroing pass in cost and duration estimates. PVE's LVM saferemove defaults to 10 MiB/s, so a 1.5 TiB disk's cleanup (~44h) can dwarf its ~2.2h mirror; disabling this only if you have verified saferemove is off on every storage in a group understates real migration cost badly if that assumption is wrong.

`migration.wipe_load_weight`

Weight, default 1.0.

In-flight I/O charged to the source for the whole saferemove wipe duration — both in the cost model and in the saturation guard, where a draining move charges this to its source and nothing to its target. The zeroing pass is one sequential writer, so 1.0 is the natural value.

`migration.saturation_ceiling`

Fraction in (0, 1], default 0.85.

The fraction of a storage's `saturation_load` (not of `capability_weight`) a move may drive it to. Inactive for any storage whose `saturation_load` is null — the default, since there is no safe way to infer it.

`migration.assume_thick_provisioning`

Boolean, only `true` (the default) is accepted. **Kept so a config that spells it out still loads; false is refused at startup.**

This tool never considers over-provisioning. Every disk counts at its *provisioned* size — in the reserve arithmetic, the free-space requirement, the transient check and the cost model — on a thin-provisioned storage (Ceph RBD, LVM-thin, ZFS) exactly as on a thick one. Thin provisioning is what lets a pool hold more provisioned bytes than it has; here it is deliberately never counted on, because a move that only fits *if the disks stay thin* is one a growing guest can turn into a full pool.

The consequence to plan for: on a thin pool the tool's idea of "used" is the sum of the disks' sizes plus foreign volumes, which can be several times what the pool reports as allocated. A `free_space.soft` of "90%" on a pool with plenty of *actually* free space can therefore show a large shortfall in `plan/explain`, and the engine will migrate disks off it. That is the intended behaviour, not a miscalculation.

The same holds at the moment of the move: the `check apply` makes just before each `move_disk` re-reads the target's volume listing and sums the volumes' provisioned sizes, never the pool's allocated figure, so it can refuse a move onto a thin pool that still looks half empty in the PVE UI. `show-load`, `plan` and `explain` print the provisioned figure as the storage's usage and, when the pool's own number differs, add it in brackets as (pool reports X allocated).

`migration.tiny_disk_bytes`

Size, default 67108864 (64 MiB).

A disk smaller than this carries zero `beta_move_count/gamma_move_bytes_per_tib` cost in the objective and zero `cost_d` in the payback model — it still counts as a scheduled move and every hard per-move safety rule (`max_single_move_duration`, the saturation guard, the transient reserve invariant) still applies to it exactly like any other move, but it needs no payback verdict and cannot make a plan fail the aggregate `payback_ratio` test. Together with `objective.kappa_vm_affinity`, this is what lets a tiny volume — an `efidisk0` var store or `tpmstate0`, both normally a few hundred KiB to a few MiB — rejoin its VM for free instead of being priced like a real migration: both device types move online on PVE 9.2, so there is no reason to price their reunion like a real migration once they are cheap enough to ignore. (IMPLEMENTATION_PLAN.md §§5.4, 7.1, 7.3, 3.6.)

The default sits comfortably above either of those and far below any disk the payback rule was written for, so a 528 KiB EFI disk always qualifies and a 64 GiB data disk never does. Set it to 0 to restore the pre-section-7.2 accounting, where every disk — however small — is charged a full migration. Setting it too

high (larger than disks you actually want cost-accounted) lets real, meaningfully-sized migrations bypass the payback safety test entirely; there is no upper bound enforced beyond ≥ 0 , so this is an operator judgement call, not a validated range.

objective — the solver's trade-off weights

The solver (and, for the terms it also uses, the heuristic backend) minimizes one weighted sum, evaluated per group:

```
alpha_spread      * (imbalance: summed deviation of each storage's utilization from the group's
target)
+ beta_move_count  * (number of migrations)
+ gamma_move_bytes_per_tib * (TiB actually migrated)
+ kappa_vm_affinity * (VM disk fragmentation, I/O-weighted – see below)
+ delta_capacity_spread * (data spread: summed deviation of each storage's fill fraction from the group's
mean)
+ reserve_violation_penalty * (reserve violation, heuristic backend only – see below)
```

The first five terms are calibrated to share one scale, in units of average in-flight I/O per storage (see `metrics.read_time_ns` above) — which is what makes the weights directly comparable: at the defaults, a migration must buy at least a 0.25-request reduction in summed imbalance just to cover its own `beta_move_count` cost, before its `gamma_move_bytes_per_tib` and `kappa_vm_affinity` costs and the separate payback test (`migration.payback_ratio` above) are even considered. Each term is explained, with its own default and worked numbers, in its own entry below. (IMPLEMENTATION_PLAN.md section 5.4 has the full derivation; section 14.3 works two competing plans through this exact formula.)

objective.spread_metric

l1 or minmax, default l1.

How imbalance is measured: l1 (sum of each storage's deviation from the group's target utilization) or minmax (only the single hottest storage). l1 is the default because minmax is indifferent to a second nearly-as-bad storage once the worst one is fixed.

objective.alpha_spread

Weight, default 1.0.

Weight on the imbalance term (the sum of each storage's deviation from the group's target utilization — see objective above for where this sits in the full objective, and why 1.0 against the default `beta_move_count`: 0.25 means a migration must buy at least a 0.25-request reduction in that sum to be worth making at all).

objective.beta_move_count

Weight, default 0.25.

Penalty per migration — the tunable implementation of “minimize the number of migrations” (IMPLEMENTATION_PLAN.md's “Requirement interpretation” note: a strict minimum would refuse a second cheap move that halves the remaining imbalance, which is not desired). Raising it trades balance quality for fewer, larger moves.

objective.gamma_move_bytes_per_tib

Weight per TiB, default 0.05.

Penalty per TiB actually migrated, biasing against moving large disks specifically (as opposed to `beta_move_count`, which only counts moves).

objective.kappa_vm_affinity

Weight, default 0.50.

Penalty per extra storage a VM’s disks are spread across, counted only **within** a group — a VM split across two groups is structural and cannot be repaired by any migration, so it is not counted — and weighted by the VM’s own I/O share of the group: $w_v = \max(1, \ell_v / \ell)$, where ℓ_v is the VM’s total load (the sum, in average in-flight I/O, of every one of its disks in the group, including disks pinned this run) and ℓ is the group’s mean load per VM (the group’s total load divided by every VM that owns a disk in the group). The floor of 1 means a below-average VM’s fragmentation is weighted exactly as configured; a VM running several times the group’s average I/O is worth correspondingly more to keep together than the configured weight alone suggests — a VM doing 2.7x the group’s average I/O, for example, is effectively weighted as if `kappa_vm_affinity` were 2.7 times higher for that VM alone. A soft preference: a strong imbalance or a capacity constraint can legitimately override it. (IMPLEMENTATION_PLAN.md section 5.4.)

objective.delta_capacity_spread

Weight, default 0.5.

Weight on data spread — the deviation of each storage’s fill fraction (bytes used / capacity) from the group’s mean fill (IMPLEMENTATION_PLAN.md section 5.3 (C7)). Bounds the share of a group’s data any single storage failure costs, and keeps peak fill uniform across the group. I/O balance stays the first priority: this defaults to half of `alpha_spread`, and acts on its own mainly when a group’s I/O is already balanced but its data is concentrated — the case gates `capacity_spread_threshold` exists to reach. 0 disables the term entirely; configuration validation warns once it exceeds `alpha_spread`, the point where data evenness starts outweighing I/O evenness in every plan comparison.

objective.affinity_counts_pinned_disks

Boolean, default true.

Whether a disk that cannot move this run (snapshot-blocked, excluded, or on a locked VM) still counts toward `kappa_vm_affinity`. Default true: such a disk still *occupies* a storage, so the VM really is spread, and its fixed location is an anchor the VM’s movable disks can be drawn back to. Note `efidisk0/tpmstate0` are *not* in this pinned set on PVE 9.2 — they move online.

Set it to false to count movable disks only. Be aware of what that costs. Take a VM with a snapshot-blocked disk on `san-a`:

- **One movable disk, on san-b.** With false the VM’s counted footprint is that one disk, so it is never “spread”, and moving it to `san-a` (reuniting the VM) scores exactly the same as leaving it or sending it to a third storage — the reunion is invisible, and the solver may pick any of them. With true the VM counts as spread over two storages until the disk joins `san-a`, and `kappa_vm_affinity` pulls it there.
- **Two movable disks, both on san-b.** With false the counted footprint is `san-b` alone; moving one of them to `san-a` to rejoin the pinned disk makes it `{san-a, san-b}` and is charged `kappa_vm_affinity` as if the VM had just been split. With true that same move is neutral, and moving both is a gain.

The pinned disk’s storage is fixed, so counting it never asks for anything unreachable: it only prefers targets where the VM already has a disk.

This default changed after 0.1.6 (it was false). If you have not set it and your cluster has pinned disks, expect plans to prefer targets where the VM already has a disk. To keep the old behaviour, set it explicitly.

objective.reserve_violation_penalty

Weight, default 1000.0.

The reserve-violation weight in the **heuristic backend**'s objective (`solver.backend: heuristic`, or `auto` falling back to it when neither a MILP backend is installed): multiplied straight into `reserve_penalty_term = objective.reserve_violation_penalty * reserve_shortfall_tib`, one of the six terms explain's objective: line prints. It is used exactly as configured — no floor, no automatic raise, no warning.

The MILP backends (`solver.backend: cpsat/cbc`, and `auto` when either is available) solve the reserve **lexicographically** instead: the reserve is fixed as a hard constraint and solved for first, before the rest of the objective above is even considered, so no weight — this one included — can trade it away. A single-stage alternative exists on paper: computing a provably-dominant penalty from the group's own load instead of taking this key at face value, with a `max(configured, computed)` floor. It is not implemented, so this key never gets raised automatically for any backend — what you set is exactly what the heuristic backend uses, and the MILP backends never consult it at all. (IMPLEMENTATION_PLAN.md section 5.3.)

solver — which backend plans

`solver.backend`

One of `auto`, `cpsat`, `cbc`, `heuristic`; default `auto`.

`auto` prefers CP-SAT (`pip install proxmox-storage-drs[solver]` – not packaged for Debian), falls back to CBC (the packaged solver path via `python3-pulp + coinor-cbc`), then the dependency-free heuristic. `cpsat/cbc` force one specific backend, failing that group's solve back to the heuristic (never a silent substitution of the *other* MILP backend) if its library is not importable or it cannot solve within `solver.time_limit_seconds` – force a specific backend only to reproduce or compare a result. `heuristic` skips the solver entirely. See `docs/internals/91-optimize.md`.

`solver.time_limit_seconds`

Duration, default 60.

Wall-clock budget for a MILP solve attempt before falling back to the heuristic. Never emits a partial/unvalidated assignment on timeout — it falls back cleanly instead.

`solver.mip_gap`

Fraction, default 0.02.

Acceptable optimality gap for the MILP solve: CP-SAT/CBC may stop once the best solution found is within this fraction of a proven lower bound, rather than solving to exact optimality. The CP-SAT backend must also convert every fractional weight and load value into an integer coefficient before solving, which introduces its own rounding error — worked out to roughly 5×10^{-4} of summed load-deviation units even for a 1,000-disk group, three orders of magnitude below this default 0.02 gap, so that rounding error cannot itself change which plan is selected or make CP-SAT and CBC disagree. (IMPLEMENTATION_PLAN.md section 5.5.)

`solver.heuristic_iterations`

Integer > 0, default 5000.

Iteration budget for the heuristic's local-search descent phase. Higher can find a better local optimum on a large group at the cost of run time. (IMPLEMENTATION_PLAN.md section 5.5.)

execution — how (and whether) moves actually happen

`execution.mode`

One of `dry-run`, `confirm`, `auto`; default `dry-run`.

`dry-run` (the default) prints the plan and changes nothing. `confirm` executes one move at a time with a prompt before each. `auto` executes unattended, subject to the concurrency and time-window settings

below. `--mode` on the command line overrides this for one run; moving *toward* less safety is logged at warning level (IMPLEMENTATION_PLAN.md section 11.3).

execution.max_concurrent_migrations

Integer $\geq$ 1, default 1.

How many moves may be in flight across the whole run at once, in `--mode auto` only (dry-run/confirm always run strictly sequentially regardless of this setting). Above 1 requires the *generalized* form of the transient reserve invariant (see `groups[].storages[].saturation_load` above for the single-move definition): several disks can land on one storage at once, and none of their sources release space until each individually completes — apply re-checks it live before launching each move. Launch order stays strictly FIFO: apply never reorders the scheduler’s own queue to keep every slot busy, so a move that cannot launch yet is waited for rather than skipped past — see docs/manual/28-apply.md’s own “Concurrent execution” section.

execution.max_migrations_per_run

Integer $\geq$ 1, default 5.

A ceiling on how many moves one invocation executes, independent of how many the plan contains — the remainder waits for the next run. `auto` mode only (dry-run/confirm execute or report the whole plan regardless); a shared budget across every group the run visits, not reset per group.

execution.max_concurrent_per_storage

Integer $\geq$ 1, default 1.

Caps concurrent moves touching one storage, counting it as either source or target, in `--mode auto` only. At the default of 1, two moves targeting the same storage serialize automatically and the generalized transient invariant collapses to its simple single-move form. Raising it should be a deliberate act on a storage with real spare headroom — two moves off the same source also means two concurrent `saferemove` wipes sharing one throttle.

execution.max_replans_per_run

Integer $\geq$ 0, default 3.

A cap on how many times one run may abandon its current plan and re-plan from newly observed state — a mismatch between the plan and reality (a VM live-migrated mid-plan, a lock appeared, a disk created or moved by someone else, another tool filling the target) is normal, but a cluster churning faster than the engine can plan is a condition for a human, not for indefinite retrying. `auto` mode only — `confirm/dry-run` report a mismatch (`replan_needed`) and stop that group’s own run for the operator to re-run by hand, rather than re-planning automatically.

When the cap is reached the run bails out and exits 0; the next run starts from freshly observed state. Set too low, ordinary churn ends runs early and moves wait for the next one; set too high, a cluster that keeps changing under the tool keeps it planning instead of waiting. 0 disables re-planning: the first mismatch ends the run. Only a *mismatch* re-plans — an error reading PVE or Prometheus fails the run outright and never counts against this cap (see 28-apply.md, “When something cannot be read”).

execution.abort_on_failure

Boolean, default true.

On a failed move, stop the run and report rather than continuing with the remaining plan. The cluster is left in a valid intermediate state either way; the next run re-plans from reality.

execution.poll_interval_seconds

Duration, default 10.

How often an in-flight `move_disk` task's status is polled.

execution.locks.wait_timeout

Duration, default 4h.

How long to wait for a VM's config lock (any non-empty value — the set is treated as open-ended and never whitelisted) to clear before applying `execution.locks.on_timeout`. Generous by default: a backup of a large VM can easily run for hours, and that is a normal, not exceptional, condition.

execution.locks.poll_interval

Duration, default 30s.

How often the lock is re-checked while waiting.

execution.locks.on_timeout

One of skip, abort; default skip.

What happens when `execution.locks.wait_timeout` elapses: skip this move and continue with the rest of the plan, or abort the run. Never “force” — that option does not exist, deliberately.

execution.locks.task_retry_limit

Integer, default 2.

The `move_disk` task *itself* can fail immediately with PVE's own `can't lock file '/var/lock/qemu-server/lock-<vmid>.conf' - got timeout` — a different lock than the config lock attribute `execution.locks.wait_timeout` waits out, momentarily still held (typically by the previous move's own `saferemove` wipe/cleanup finishing for the same VM) even after that attribute already reads clear. This many extra attempts are made, each preceded by `execution.locks.task_retry_backoff`, before the move is reported failed like any other task failure. 0 disables the retry.

execution.locks.task_retry_backoff

Duration, default 15s.

How long to wait between a task-lock-timeout failure and the next `move_disk` retry attempt.

execution.source_release.wait

Boolean, default true.

Wait for the source volume of a completed move to actually disappear (and the VM to be unlocked) before considering the move done, rather than trusting the `move_disk` task's own OK status — with `saferemove` enabled, OK does not mean the source space is back yet. Set false only on storages verified not to wipe.

execution.source_release.timeout

Duration, default 48h.

Bound on the wait above, sized for a multi-TiB disk wiping at the default 10 MiB/s. Must be at least `largest_disk / |saferemove_throughput|` for every storage where `saferemove` is on — `verify-storages` checks this.

execution.time_windows[].days

List of mon..sun, default: every day.

Which days a time window applies on. Applies only to `execution.mode: auto` — dry-run and confirm are not time-restricted, since neither changes anything without a human already present.

`execution.time_windows[].start / execution.time_windows[].end`

HH:MM strings, required together, must differ.

The window auto mode is permitted to execute moves in, in the **local time of the host running pve-storage-drs** — the same convention `systemd.timer`'s own `OnCalendar=` uses by default — not UTC. A window may cross midnight (`start: "22:00", end: "06:00"`); `start == end` is rejected as ambiguous — it would silently mean either “never” or “always”. No `time_windows` configured at all means no restriction: auto may execute at any time — this is the deliberate default, not merely the absence of a rule. Before starting a move, auto also refuses it (and stops the group's run cleanly, without aborting a move already in progress) if its estimated duration would not finish before the window closes.

exclude — what DRS never touches

exclude.vmsids

List of integers, default [].

VM ids whose disks are never moved.

exclude.disks

List of "vmid:device" strings, default [], e.g. ["101:scsi1"].

Individual disks excluded regardless of their VM.

exclude.storages

List of storage ids, default [].

Storages excluded as both source and target — effectively removes them from their group without editing groups.

exclude.tags

List of PVE tags, default ["no-drs"].

A VM carrying any of these tags opts out entirely.

exclude.skip_vms_with_snapshots

Boolean, default true.

Pre-filter disks with a snapshot or an unreferenced companion volume before they reach the API. Setting this false does **not** make such a move work — PVE rejects `move_disk delete=1` on a snapshotted volume regardless; it only removes the early, informative warning. Keep this true.

exclude.running_only

Boolean, default true.

Only running VMs generate load to balance; a stopped VM's disks are moved only if a capacity constraint requires it. Setting this false widens the movable set to stopped VMs' disks for capacity purposes but they still contribute zero load.

exclude.include_unused_disks

Boolean, default true.

Whether detached (unusedN) volumes are movable. They carry zero load, so the solver only relocates one to repair a capacity violation — never for balance, since there is no benefit to buy. Set false to pin them in place entirely; they still count via `snapshot_reserve.count_foreign_volumes` either way.

report**report.warn_pinned_load_fraction**

Fraction in (0, 1], default 0.25.

If pinned load (snapshots, locks, exclusions) exceeds this fraction of a group's total, the residual imbalance may be structural rather than a planning failure, and `explain` says so alongside the best achievable spread given the pins — see [29-explain.md](#).

state**state.path**

File path, default `/var/lib/pve-storage-drs/state.json`.

Local disk, one copy per host, deliberately never `/etc/pve` — see [00-installation.md](#). Losing this file is safe but not free: cooldowns and the drift baseline reset, so the next run may migrate sooner than intended. Treat it as state to back up, not as a cache.

`show-load` and `plan` both read this file (if present) for the drift gate's history; neither writes it, and a missing or unreadable file just means every group evaluates as if it had never been balanced before — see [../internals/15-state.md](#).

forecast — history beyond the plain quantile

See `IMPLEMENTATION_PLAN.md` section 10 and `proxmox_storage_drs/forecast.py`.

forecast.model

One of `quantile`, `seasonal_naive`, `holt_winters`; default `quantile`.

`quantile` needs only `window.lookback` and no model fitting. `seasonal_naive` and `holt_winters` need more history than the decision window alone — see below — and `pve-storage-drs` refuses to start if `window.lookback` (or your Prometheus retention) cannot supply it, rather than silently falling back.

Backtest-validated before use. Only when the saturation guard is actually active (some `groups[].storages[].saturation_load` is set): a `seasonal_naive/holt_winters` model is fit on the older half of its own recent history and checked against what actually happened in the newer half, once per group, before it is trusted for that run. A model that misses by more than `gates.imbalance_threshold` — or that does not yet have enough history to backtest at all — falls back to `quantile` for that group's saturation guard this run, logged at warning. `quantile` itself is never backtested; there is nothing to validate and nothing more conservative to fall back to. (`IMPLEMENTATION_PLAN.md` section 10.2.)

forecast.seasonal_lookback_days

Days, default 7.

History `seasonal_naive` needs: same-hour-of-day samples across this many days. 0 does not disable the model or fall back to anything — it simply stops requiring history beyond `window.lookback`, which for a `lookback` under 24h can leave no same-hour-of-day sample to match at all (predicting 0.0, not a fallback).

forecast.holt_winters.seasonal_periods

Integer > 0, default 288 (24h at a 5-minute step).

Samples per seasonal cycle. Combined with `metrics.step`, this determines `holt_winters`'s required history: $2 * \text{seasonal_periods} * \text{metrics.step}$ — 48h at the defaults, which a 24h `window.lookback` can never satisfy.

To fit **weekly** seasonality instead of daily (e.g. weekends look different from weekdays), widen this together with `window.lookback`: `seasonal_periods: 2016` (7d at the default 5m step) needs `window.lookback` of

at least 28 days ($2 * 2016 * 5m$). The fetch behind this — both the live plan/apply saturation guard and `collect-testdata` — is chunked into day-sized requests regardless of how wide the range gets, so a large `seasonal_periods/window.lookback` combination no longer risks exceeding a Prometheus-compatible backend's own per-series resolution limit (VictoriaMetrics/gigapipe's default 11,000 points) the way an unchunked request over the same range once could; it costs more Prometheus requests per run instead (one per day of range, per raw metric, per group) — a real cost worth sizing deliberately, not a reason to keep the range artificially small.

`forecast.holt_winters.trend`

One of add, mul, none; default add.

The trend component passed to the underlying Holt-Winters fit (`statsmodels`, an optional dependency — see [30-safety-and-status.md](#)).

`forecast.holt_winters.seasonal`

One of add, mul, none; default add.

The seasonal component passed to the same fit.

`forecast.holt_winters.residual_z`

Weight, default 2.0.

The upper bound is `point_estimate + residual_z * stdev(residuals)` from the in-sample fit — this is what the saturation guard actually consumes (see `window.upper_quantile` above for the equivalent under the quantile model; the optimizer itself does not consume this).

support — diagnostic bundles

See [26-collect-testdata-and-replay.md](#) and `IMPLEMENTATION_PLAN.md` section 16. Nothing here is read on the normal plan/apply path — only by `collect-testdata` and by `--replay`'s config loading.

`support.salt_path`

File path, default `/var/lib/pve-storage-drs/anonymization-salt`.

32 random bytes, generated on first use and persisted here, mode 0600, never written into a bundle. Every pseudonym `collect-testdata` produces is HMAC-SHA256 keyed on this salt, so the same object maps to the same pseudonym across runs and across bundles until `--new-salt` rotates it. Reproducibility is per salt file, i.e. per node by default; point this at a `pmxcfs` path (e.g. `/etc/pve/pve-storage-drs-anon-salt`) if every node in the cluster should agree on one mapping — that also replicates the salt to every node, which is the trade-off being made.

`support.bundle_dir`

Directory path, default `/var/lib/pve-storage-drs/testdata`.

Default `-o/-` output for `collect-testdata` when the flag is not given.

`support.max_series_points`

Positive integer, default 5000000.

`collect-testdata` prints the estimated series-sample count before fetching anything and refuses outright above this ceiling — never a silent truncation — naming the `--range/--step/--no-series` flags that would bring the estimate under it.

`support.capture_range`

The literal string "auto", or a duration, default "auto".

"auto" captures the union of every forecaster's `required_range()` — currently `max(window.lookback, forecast.seasonal_lookback_days, 2 * forecast.holt_winters.seasonal_periods * metrics.step)` — so a bundle can reproduce a forecaster the capturing operator never configured. An explicit duration (e.g. 14d) overrides that; `--range` on the command line overrides both.

monitoring — telling your monitoring system what the last run did

`monitoring.status_file`

String (a file path) or `null`, default `null`.

Where `apply` leaves its status report. `null` writes nothing. Set, the file is rewritten at the end of **every** `apply` run that produced a result — dry-run included, and runs that moved nothing included — in the format the `check_statusfile` plugin (Debian package `monitoring-plugins-contrib`) reads, so a plain `check_statusfile <path>` check is all a Nagios-style monitoring system needs. See [36-monitoring.md](#) for the format, what makes a run OK, WARNING or CRITICAL, and how to wire the check up.

Choose a path the user `apply` runs as can write and the monitoring user can read (the file is mode 0644; the parent directory is created if it is missing). `/var/lib/pve-storage-drs/status` beside `state.json` is the natural place.

It interacts with the plugin's `--age`: the file's modification time is what says the timer is still firing, so a timer that runs every 15 minutes wants a check like `check_statusfile -a 1h`, while the plugin's default of 26 hours suits a daily one. Set too generous an age and a stopped timer goes unnoticed for that long; too tight and one slow run turns the check WARNING. A write failure is logged (`status_file_write_failed`) and never changes the run or its exit code.

Verifying your setup

Run this before relying on any plan. Telegraf and InfluxDB naming varies by deployment, so `pve-storage-drs` never assumes the defaults in [10-configuration.md](#) are correct for your cluster — `verify-metrics` is what checks them against the live Prometheus instead.

It also checks something the names alone cannot tell you: whether all six counters actually survived the transport. See [05-metrics-pipeline.md](#) for why they might not have, and why that produces a wrong plan rather than an error.

Running it

```
pve-storage-drs -c /etc/pve/drs.yaml verify-metrics
```

A successful run looks like this (labels and values are illustrative):

```
[ info] metrics.read_ops = 'blockstat_rd_operations' exists
[ info] metrics.write_ops = 'blockstat_wr_operations' exists
[ info] metrics.read_bytes = 'blockstat_rd_bytes' exists
[ info] metrics.write_bytes = 'blockstat_wr_bytes' exists
[ info] metrics.read_time_ns = 'blockstat_rd_total_time_ns' exists
[ info] metrics.write_time_ns = 'blockstat_wr_total_time_ns' exists
[ info] blockstat_rd_operations: sample series labels {'vmid': '101', 'instance': 'scsi0', 'nodename':
'pve01', 'cluster': 'pvezebe'}
...
[ info] 'cluster' label values seen across these metrics: pvezebe
...

OK: verify-metrics found no blocking problems
```

Exit status is 0 when every check passes, 1 when any check reports an error-level finding (see below).
 - -json emits the same information as one JSON object instead — see [30-safety-and-status.md](#) for exit codes in general.

What each finding means

verify-metrics runs six checks, in this order (IMPLEMENTATION_PLAN.md section 3.3):

1. **Metric names exist.** Each of `metrics.read_ops`, `write_ops`, `read_bytes`, `write_bytes`, `read_time_ns` and `write_time_ns` is looked up against Prometheus's own `/api/v1/label/___name___/values`. A missing name is an **error** — fix the name in `10-configuration.md`'s `metrics.*` section to match what your Telegraf actually emits.
2. **Sample series and labels.** One instant query per metric, printing its full label set so you can see at a glance whether `metrics.labels.vmid`, `.device` and `.node` are really present and non-empty on a live series. A metric that exists but currently has no series is a **warning**, not an error — it may simply mean nothing has generated that kind of I/O recently. This step also cross-checks the full (`vmid`, `device`) set each of the six metrics reports against the other five — at no extra Prometheus cost, since it reuses these same six instant queries. If one metric is silently missing a disk the other five all report, that is a **warning** naming the metric and the disk: the usual cause is Telegraf's Prometheus-compatible output dropping a field the moment it sees a non-numeric value for it (InfluxDB's line protocol fixes a field's type from its first write, and Prometheus/OpenMetrics has no string sample type) — independently of the other five fields for the same disk, which is exactly why check 5 below (only looking at `read_ops`) cannot catch this on its own if a *different* field is the one affected.
3. **Configured labels present.** If `metrics.labels.vmid/.device/.node` do not appear (or are empty) on the sample series, that is an **error**: the load model has nothing to join disks on.
4. **The instance collision.** If `metrics.labels.device` is still the literal string instance, a **warning** — PVE's own per-disk tag has this name and it collides with Prometheus's unrelated scrape-target instance label. Many Telegraf configurations rename or overwrite one of the two; confirm which one survived before trusting this.
5. **Per-disk coverage.** For every (`vmid`, `device`) pair seen on `metrics.read_ops`, the fraction of expected samples actually present over `window.lookback` is measured. Below `window.min_coverage`, a **warning** naming the disk — that disk's own data will be rejected by the load model at plan time and its last known load from `state.json` used instead.
6. **Observed sample spacing.** The modal gap between consecutive samples of a live series is measured directly, rather than trusted from `metrics.pvestatd_push_interval`. Disagreeing by more than 20%, or `metrics.rate_window` being below four times what was actually observed, is an **error** — this is what stops a stale `pvestatd_push_interval` declaration from silently breaking the `rate()` window it is supposed to protect.

If it fails

- **A metric does not exist:** check your Telegraf `influxdb_listener` (or equivalent) configuration for the actual measurement/field names it is producing, and update `metrics.*` to match.
- **A label is missing or empty:** the sample series printed by check 2 is the fastest way to see what labels Prometheus actually has for that series — update `metrics.labels.*` to name the real ones.
- **One metric is missing disks the others report:** a transport dropped a field. Check your Telegraf configuration (and the PVE-side field it collects) for that specific field emitting a non-numeric value for that disk at some point — that is enough to make Telegraf drop it permanently for that series, even while every other field for the same disk keeps working normally. This is a real defect in the pipeline, not noise, and it will not resolve itself; [05-metrics-pipeline.md](#) explains the mechanism, which output plugins are prone to it, and the gigapipe-on-ClickHouse setup that avoids the format

bridge entirely.

- **Coverage is low for specific disks:** check that Telegraf is actually scraping every node, and that `window.lookback` is not longer than your Prometheus retention.
- **Spacing disagrees:** measure `pvestatd`'s real push interval on a node (its `systemd` timer, or the interval visible in `/etc/pve/status.cfg` if set there) and correct `metrics.pvestatd_push_interval`.

Reading show-load and verify-storages

Both commands only read — neither ever changes anything, in any `execution.mode`. Run them any time you want to see what the tool sees.

show-load

Prints every storage in every configured group, its disks, and its snapshot-reserve status. This example uses the group from `IMPLEMENTATION_PLAN.md` section 14's worked example (`config/drs.example.yaml` ships the same group and weights) so the numbers are traceable to that section rather than invented for this page:

```
$ pve-storage-drs -c /etc/pve/drs.yaml show-load
Group fc-tier1 → ACT: reserve violated on san-a; acting now regardless of the normal drift/imbalance
thresholds -- a capacity shortfall is never delayed by them
  san-a  provisioned 4.50 TiB/8.00 TiB  L=6.50 u=6.50  ▲ reserve short by 512.00 GiB  (largest disk 2.00
  TiB, requires 4.00 TiB free)
    web01(101):scsi0          2.00 TiB  raw    ℓ 3.00
    web01(101):scsi1          1.00 TiB  raw    ℓ 1.00
    db01(102):scsi0           1.50 TiB  raw    ℓ 2.50
  san-b  provisioned 1.50 TiB/8.00 TiB  L=0.70 u=0.70  reserve OK  (largest disk 1.00 TiB, requires 2.00 TiB
  free)
    mail01(103):scsi0         512.00 GiB  raw    ℓ 0.40
    app01(104):scsi0          1.00 TiB  raw    ℓ 0.30
  san-c  provisioned 512.00 GiB/8.00 TiB  L=0.20 u=0.20  reserve OK  (largest disk 512.00 GiB, requires 1.00
  TiB free)
    cache01(105):scsi0        512.00 GiB  raw    ℓ 0.20
```

provisioned on a storage's line is the sum of every disk's *provisioned* size on it (the managed disks, plus any foreign volumes — templates, ISOs, backups, other groups' disks) against the storage's capacity — the figure the reserve shortfall beside it is computed from. The tool never counts on over-provisioning: a thin-provisioned pool (Ceph RBD, LVM-thin, ZFS) is counted at what its disks were provisioned at, not at what happens to be allocated. On a thick pool the two are the same number. On a thin one the pool's own, lower figure is printed next to it so the shortfall does not look like a mistake:

```
ceph-a  provisioned 3.36 TiB/9.48 TiB (pool reports 1.45 TiB allocated)  L=2.10 u=2.10  reserve OK
(largest disk 1.00 TiB, requires 2.00 TiB free)
```

Only the provisioned figure ever enters a decision; the allocated one is shown for information. (`--json` carries both: `provisioned_used_bytes` and `used_bytes`.)

$L=$ / $u=$ on a storage's line are L_s — the sum of ℓ over every disk currently on that storage — and $u_s = L_s / \text{capability_weight}$, the *utilization* that actually gets balanced across storages (so a storage configured with half the `capability_weight` of another is correctly expected to carry half the load, not the same absolute amount). ℓ after a disk's size/format is that disk's own load, measured in **average in-flight I/O requests**: by default it is $\text{rate}(\text{rd_total_time_ns} + \text{wr_total_time_ns}) / 1e9$ for that disk — how many read/write requests are, on average, in progress on the device at once — which self-weights by cost: an 8 ms write and a 1 ms read are correctly counted eight-to-one rather than as equal operations, the way a plain IOPS count would. (Read and write can be weighted asymmetrically via `load_weights.read_factor/write_factor`, and ops/bytes terms can be blended in alongside I/O time; see `IMPLEMENTATION_PLAN.md`

section 4 for the full normalization and docs/internals/70-loadmodel.md for the code.) The worked numbers above are IMPLEMENTATION_PLAN.md section 14's own example (config/drs.example.yaml ships the same group and weights), not invented for this page.

The Group <name> → ACT/NO ACTION line

Four gates, evaluated in this order — **reserve override**, then the **capacity gate**, then **drift**, then **imbalance** — and the first one that decides wins. The header line above shows the reserve override: san-a's 0.5 TiB shortfall forces ACT outright, regardless of how balanced or drifted the group is, because “safety is not subject to hysteresis” (section 13). The other shapes this line can take, on a group with no reserve violation:

```
Group fc-tier1 → ACT: imbalance 255.4% meets or exceeds gates.imbalance_threshold (20.0%)
Group fc-tier1 → NO ACTION: imbalance 12.2% is below gates.imbalance_threshold (20.0%)
```

The **capacity gate** is the data-spread counterpart of the imbalance gate: instead of comparing I/O load across a group's storages, it compares how *full* each one is, as a fraction of its own capacity. It fires — bypassing drift and imbalance the same way the reserve override does, on the reasoning that a quiet workload is not a reason to leave data lopsided across storages — when the spread in fill fraction across the group's storages is at least gates.capacity_spread_threshold (default 0.25) relative to the group's mean fill:

```
Group fc-tier1 → ACT: capacity spread 32.0% meets or exceeds gates.capacity_spread_threshold (25.0%) --
acting now regardless of I/O drift/imbalance
```

Unlike the other three, this gate can act on a group whose I/O is already perfectly balanced — objective.delta_capacity_spread is what does the actual spreading once the solver runs. Setting gates.capacity_spread_threshold: null disables it outright, and it never fires on a group with no data at all (mean fill 0).

show-load reads state.path's recorded load vector (section 11.2) and passes it to the drift gate, so once a group has one on record, this line can also read:

```
Group fc-tier1 → NO ACTION: drift 3.1% is below gates.drift_threshold (10.0%)
```

A group with no state.json, or none recorded for it yet, still evaluates as if this were the very first run — the drift gate is skipped outright (section 6's own degenerate-case rule), not “treated as zero drift” — so its verdict falls straight through to a reserve override, the capacity gate (which never depends on state.json at all), or an imbalance check, exactly as before this was wired up. docs/internals/80-gates.md and docs/internals/15-state.md have the detail; apply writes last_balance once a run actually executes at least one move (confirm or auto mode – dry-run only simulates, so it never triggers this), so this first-run behaviour is only the common case for a group before its first successful migration – after that, show-load's drift line reflects real history.

A pinned disk carries [pinned: <reason>] after its size and format — snapshots present (N), pending config change (unapplied), pending deletion (unapplied), locked: <lock>, excluded by config, excluded: unused disk (exclude.include_unused_disks=false), or cooldown: moved recently, <time> left on gates.cooldown_per_disk whenever state.json records that disk having moved within gates.cooldown_per_disk — matching IMPLEMENTATION_PLAN.md section 5.3 (C2) exactly; that reason is the answer to “why won't it move this disk.” The two pending reasons (section 3.8) mean PVE has an edit or a deletion queued against that disk key that has not taken effect yet (usually waiting on a VM reboot) — apply or revert it in the PVE UI, then re-run.

A disk whose measured I/O falls below window.min_coverage (section 3.4) gets a ▲ <disk> : <reason>

line of its own, right after that group’s disks, instead of a silently wrong ℓ — it either shows a `state.json`-recorded last known load (see `docs/internals/70-loadmodel.md`) or `0.0` when no `state.json` entry exists for it, and either way the warning says which. `tpmstate0` and `unusedN` disks are never flagged this way: they genuinely emit no I/O metrics, so $\ell\ 0.00$ for them is correct, not a gap. A group with no measured I/O at all gets one (`idle: no measured I/O for this group this window`) line instead of per-disk warnings — unless every one of the six queries came back with literally zero series (as opposed to some series with a genuinely-zero rate), in which case the line reads Δ the resolved query filter matched no series at all instead: that is a scoping problem (`metrics.labels.node`, or `extra_selector`), not an idle cluster, and `verify-metrics`’s sample-series labels (`docs/manual/20-verifying-metrics.md`) are the next place to look — `REVIEW.md W-06/W-07`.

A Prometheus outage does not fail this command. Sizes and reserve status never depend on Prometheus at all; if the load fetch for a group fails, that group’s storage/disk lines simply omit $\ell=$ / $u=$ / ℓ , and a Δ per-disk load unavailable: `<reason>` line explains why — check `verify-metrics` first if you see this.

A Warnings: block, when present, lists things worth a look but not fatal: a disk on a storage that is not in any configured group (“ungrouped, not managed” — check whether that storage belongs in a group), or a disk whose size came from its own config rather than the storage’s authoritative content listing. The latter usually means the volume no longer exists on the storage — most often because it was removed directly on the storage backend rather than through Proxmox, which leaves a dangling reference behind that PVE does not clean up or even notice on its own (a VM starts fine with one; see `IMPLEMENTATION_PLAN.md` section 3.6 for a confirmed case). If you see this warning, check whether the named volume still exists on the storage and, if it genuinely does not, remove the stale reference (`qm unlink <vmid> <device>` for an `unusedN` entry).

`--json` emits the same information as one object with `groups[].storages[]` (including the exact byte counts behind the reserve check, plus load and utilization when available) and `groups[].disks[]` (plus load and `load_flagged_reason` when available), with each group carrying its own `load_computed`, `idle`, `load_error` and `gate` fields — a Prometheus outage on one group never prevents another group’s `load_computed: true` or `gate` from being reported. `gate` is `null` when `load_computed` is `false` (no `GroupLoad` to evaluate gates against) and otherwise an object with `act` (bool), `reason` (string, identical to the human line’s text after the arrow), `reserve_override` (bool), and `drift_fraction/imbalance_fraction/capacity_fraction` (float or `null` — `null` means that gate was never reached, not that it evaluated to zero; `capacity_fraction` is the capacity gate’s own ratio described above — `null` whenever `gates.capacity_spread_threshold` is unset or the group’s mean fill is 0, the same two cases in which the gate itself never fires).

A config with several groups issues Prometheus queries per group. Computing one group’s load takes seven queries (six raw metrics plus one coverage check), each scoped to that group’s own VMs — a config with `N` groups still makes `7N` queries in total on every `show-load`, not 7, since each group’s coverage and data-quality decisions are independent and so cannot share a single fetch. For the typical one-to-three-group deployment this is a handful of fast instant queries and not worth worrying about; an operator running many groups against an already-busy Prometheus should be aware of the multiplier. Each of those queries only aggregates the VMs the calling group actually has (split into batches of `metrics.VMID_QUERY_BATCH_SIZE` VMs for a single very large group) rather than every VM in the cluster — on a cluster with hundreds of VMs, an earlier version of this tool aggregated the whole cluster on every one of those `7N` queries, which could make Prometheus itself time out the query outright rather than merely running slowly.

verify-storages

Reports the resolved `free_space.soft/.hard` bytes for each storage, each with the level it came from (IMPLEMENTATION_PLAN.md section 5.3.1) — the derivation an operator cannot otherwise predict, once inheritance, `/.../` patterns, percentages and the deprecated `snapshot_reserve.min_free_bytes` fold are all in play — plus `saferemove` and the wipe time it implies for the largest disk on each storage, warning when your configured cooldown or move-duration limits are shorter than that implied wipe — the condition IMPLEMENTATION_PLAN.md section 9.3 describes as “the next run plans onto a storage that is still draining”. Section 14’s fixture has `saferemove` off everywhere (stated explicitly there, so its payback arithmetic is reproducible); this example instead assumes an LVM `san-b` with `saferemove` on at PVE’s default 10 MiB/s throughput, to show what the warning looks like:

```
$ pve-storage-drs -c /etc/pve/drs.yaml verify-storages
Group fc-tier1
  san-a saferemove=off
    free_space: soft=0 B (global) hard=0 B (= soft (no dip))
    saferemove is off or throughput unknown; no wipe-time check
  san-b saferemove=on
    free_space: soft=0 B (global) hard=0 B (= soft (no dip))
    implied wipe time for the largest disk (1.00 TiB): 1.2d
    ▲ gates.cooldown_per_storage (1.0h) is shorter than the implied wipe time -- the next run may plan onto
    a still-draining storage
    ▲ migration.max_single_move_duration (6.0h) is shorter than the implied wipe time -- a move of the
    largest disk would be rejected outright
  san-c saferemove=off
    free_space: soft=0 B (global) hard=0 B (= soft (no dip))
    saferemove is off or throughput unknown; no wipe-time check
```

`free_space.soft/.hard` read 0 B here because the fixture sets no `free_space` knob at all — the pre-section-5.3.1 default, unchanged. A config with a non-zero `free_space.soft` (or the deprecated `snapshot_reserve.min_free_bytes`, folded in) shows the resolved byte count here, the same value `plan/explain/show-load` all enforce.

The parenthesis after each value says where it came from, which is the part you cannot read off the config file when one line can mean a different number per LUN:

Shown	Meaning
global	inherited from the top-level <code>free_space</code> block
storage entry	written on this storage’s own <code>groups[] . storages[]</code> entry
pattern <code>/re/</code>	supplied by the <code>/.../</code> pattern entry that matched this storage
<code>..., 10% of 30.00 TiB</code>	appended when the value was a percentage: the capacity it was converted against
folded from <code>snapshot_reserve.min_free_bytes</code>	(soft only) the deprecated key was larger than the value written, so it raised the floor
<code>= soft (no dip)</code>	(hard only) <code>hard</code> is null, so it equals the resolved soft — including a folded one

`- - json` carries the same strings as `free_space_soft_source` and `free_space_hard_source` beside the two byte counts.

Storage types without `saferemove` at all (Ceph RBD, ZFS) always report `saferemove=off` and skip the check — there is nothing to wipe.

A negative `saferemove_throughput` is normal

If your storage.cfg reads

```
saferemove_throughput -1073741824
```

that is **1 GiB/s**, not an error and not a negative rate. PVE passes the value straight to `cstream -t`, where the sign chooses how the limit is enforced and the magnitude is the rate:

- **positive** — a session average. The wipe may exceed the rate for a while to make good on earlier underutilization, so that the run as a whole converges on it.
- **negative** — an upper limit on each individual read/write, never exceeded.

`verify-storages` reports the implied wipe time from the magnitude, so `-1073741824` and `1073741824` give the same duration; the `--json` output's `saferemove_throughput_bytes_per_sec` still echoes the value exactly as PVE has it, sign included, so you can compare it against `storage.cfg` without arithmetic. Everything that consumes the number — the migration cost the payback test charges, the `migration.max_single_move_duration` check, `apply's` source-release wait — reads it the same way.

Versions up to 0.1.6 divided by the signed value instead, which produced a *negative* wipe time (implied wipe time for the largest disk (1.00 TiB): `-17.1m`). If you are on one of those, the two warnings above could never fire on a storage configured this way, and — where `|saferemove_throughput|` happens to equal `migration.bwlimit_bytes_per_sec`, a natural thing to configure — every move's total duration came out as exactly zero, so `migration.max_single_move_duration` was not enforced at all. Upgrading is the fix; no configuration change is needed.

If any `groups[].storages[].id` is written as a `/regex/` pattern (IMPLEMENTATION_PLAN.md section 11.4), `verify-storages` also prints what each one matched this run, and lists any cluster storage matched by no group at all — the same “ungrouped, not managed” visibility `show-load` gives per disk, but at the storage level and independent of whether a disk currently happens to be on it:

```
Pattern expansions:
[fc-tier1] /san-.* / → san-a, san-b, san-c

Cluster storages matched by no group:
- local-only
```

An over-broad or dead pattern is the one new way to misconfigure a group this feature adds, so this expansion is never silent: every run also logs it at INFO, whether or not `verify-storages` is the command being run.

Diagnostic bundles: collect-testdata and --replay

IMPLEMENTATION_PLAN.md section 16 is the specification; this page is the operational side of it. Two facts motivate the whole feature: one cluster is not a test corpus (every interesting bug so far was found by an operator running the tool on a cluster the author has never seen, then *describing* what they saw), and the data cannot leave the cluster as-is (a PVE inventory is customer names, hostnames, IQNs, ticket numbers and credentials).

collect-testdata: capturing a bundle

```
pve-storage-drs -c /etc/pve/drs.yaml collect-testdata
```

This is **read-only and dry-run-only** by construction: it never issues `move_disk`, and `--mode confirm/--mode auto` alongside it is a usage error (exit 2), not a silently ignored flag. It captures the *superset* of what any supported configuration could need — every raw metric over the union of every forecaster's own history requirement, not just the one `forecast.model` you have configured — so a bundle captured today can still reproduce a Holt-Winters misfit tomorrow.

--estimate sizes the capture first: it reads the full PVE inventory (the same read pass a real capture or a plan run makes — every VM config, every content listing, every status call) to size the series capture, but fetches nothing from Prometheus:

```
pve-storage-drs -c /etc/pve/drs.yaml collect-testdata --estimate
```

```
groups: 2   disks: 47
range: 7d   step: 5m
estimated Prometheus queries: 117
estimated series sample points: 1137024
```

(The query count includes the day-sized chunking --estimate itself performs at capture time — 7 range-query chunks per group per metric at the 7 d default, not 1 — so it is the real HTTP request count, not a logical-query count that understates it.)

The sample-point figure is $47 \text{ disks} \cdot 6 \text{ metrics} \cdot 4032 \text{ points}$, not the 2016 a plain $604800s / 300s$ would suggest: at `metrics.step == metrics.rate_window` (both 5m here, and the project's own default) a live capture issues, and stores, range queries at *half* the configured step to work around a gigapipe backend bug (docs/internals/30-metrics.md), doubling the points actually captured. --estimate accounts for this so the printed figure — and the `support.max_series_points` refusal below, which compares against the same number — matches what a real capture stores, not the configured step's naive arithmetic.

Above `support.max_series_points` (default 5,000,000) the real capture refuses outright, naming the flags that bring it under the limit:

Option	Effect
-o, --output DIR	Where the bundle directory and its .tar.gz are written (default <code>support.bundle_dir</code>).
--estimate	Read the full PVE inventory (one planning run's worth of API calls), print the estimate above, and exit; fetches nothing from Prometheus.
--range DURATION	Override the series capture range (default <code>support.capture_range</code>).
--step DURATION	Override the series resolution (default <code>metrics.step</code>).
--no-series	Topology, instant queries and findings only — no per-disk time series, so the bundle cannot exercise a seasonal forecaster, but is much smaller.
--no-archive	Write the directory only, skip the .tar.gz.
--salt-file PATH	Use a different anonymization salt (default <code>support.salt_path</code>).
--new-salt	Generate a fresh salt first. Logged at warning level: bundles made before and after no longer share a pseudonym mapping.

Exit status is 0 when every capture step succeeded, 1 when the bundle was still written but the manifest records one or more failed calls (a `Datastore.Allocate`-shaped permission gap looks exactly like this — see 00-installation.md), 2 on a usage error.

What is in a bundle, and what is not

A bundle is a directory (manifest.json, config.yaml, findings.json, pve/, prometheus/, SHA256SUMS) plus a deterministic .tar.gz of it. **Read it before sending it anywhere** — it is plain, sorted, indented JSON, meant to be opened in an editor:

- **No direct identifier and no secret.** `config.yaml` never carries `proxmox.host/auth`, `prometheus.url`, or any other credential or endpoint — they are dropped, not blanked. Every node name, storage id, vmid, volume id, tag, pool and UPID is replaced by a pseudonym keyed

to a salt (`support.salt_path`) that is generated locally, never written into the bundle, and never leaves the host. Free text — VM names, descriptions, snapshot names, storage comments — is dropped outright. Device keys (`scsi0`, `efidisk0`, ...) are not identifiers and are kept as-is, since the movability rules read them directly.

- `manifest.json` carries the **PVE and Prometheus server versions** (`capture.pve_version/capture.prometheus_version`, null if that one call failed) — not identifiers, so they need no anonymization, and they are the one piece of “what was this captured against” a bundle’s own files otherwise leave to `tests/corpus/*.submission.yaml`, which only the committed corpus carries.
- **The mapping is stable per salt**, so a second bundle from the same host names the same objects the same way — useful for comparing a before/after pair — until `--new-salt` rotates it.
- **Sizes, capacities, load shapes and the whole configuration (minus credentials) are preserved.** That is what makes a bundle a test case instead of a shape. It also means a bundle is **not anonymous against someone who already knows the cluster** — a cluster with three nodes, two 8 TiB LUNs and 1,214 VMs is recognizable to anyone who has seen it. Submitting one to the project (see `tests/corpus/README.md`) is an informed decision, not a formality.

`--replay`: running against a bundle offline

```
pve-storage-drs --replay ./drs-testdata-cluster-3f8a91c2 plan --json
```

`--replay PATH` is a **global** option, accepted before any subcommand, and substitutes both the PVE and Prometheus clients at the one place each is constructed — so `verify-metrics`, `verify-storages`, `show-load`, `plan` and `explain` all run **unchanged**, with **no network access at all**. `PATH` is the **unpacked bundle directory**, not the `.tar.gz` it was sent as — `tar -xzf drs-testdata-cluster-3f8a91c2.tar.gz` first.

- The configuration is the bundle’s own `config.yaml`, unless `-c` is also given — that is how you run the operator’s cluster through a solver backend, a `spread_metric`, or a forecaster they never selected themselves.
- **apply is refused**, and so is any `--mode` above `dry-run` — exit 2 either way. A replay can never issue a write.
- A query the bundle has no recorded response for is a **loud, specific error** naming the query and the range it asked for — never a silent empty result standing in for “no data”:

```
pve-storage-drs: bundle has no recorded response for the range query
'sum by (vmid, instance) (rate(blockstat_rd_operations{...}[300s]))'
over [1700000000,1700086400] -- captured range was [1699400000,1700000000]
```

This is the failure mode to expect if you hand-edit a bundle’s `config.yaml` to a `window.lookback/metrics.step` combination the capture never anticipated, or if `--step` was overridden at capture time to something a replayed forecaster does not expect.

- `state.json` is read from the bundle if present, and never written — a replay never touches the host’s real state.

Run `--replay` against your own bundle before sending it anywhere: “does `plan` against this bundle show the problem I am reporting?” is exactly the question it answers, and it is also how the author verifies a submitted bundle reproduces what its `tests/corpus/*.submission.yaml` claims.

Sending one to the project

Bundles are wanted, and a pull request carrying one is as valuable as a pull request carrying code. A bug report describes a failure; a bundle reproduces it offline, on a machine that has never seen your

cluster, for as long as the project exists. It is also the only thing that can prove the MILP solver and the fallback heuristic still agree at a few hundred disks — the synthetic fixtures in `tests/fixtures/` top out at six.

The terms, in short:

- **Read the bundle first.** Not the audit's word for it — your own. The scrub audit assumes the collector has a bug, which is the right assumption to make about a privacy control, but a clean audit is not consent.
- A bundle under **8 MiB unpacked** can be committed to the repository; larger ones stay outside it and are exercised from a directory named in `DRS_CORPUS_DIR`.
- It needs a `<bundle-name>.submission.yaml` recording who collected it, what it reproduces, that you reviewed it, and that you agree to it being redistributed under AGPL-3.0-or-later. Accepting one makes it public permanently, in git history.
- If your cluster is one whose size and shape alone are confidential, send a `--no-series` bundle, or just a description. A description is still worth having.

`tests/corpus/README.md` in the source tree (<https://github.com/bzed/proxmox-storage-drs>) is the full guide: the submission template, the four passes the suite runs over every committed bundle, and the consent terms in full.

Reading plan

`plan` computes what `pve-storage-drs` would do and prints it. It never changes anything, in any `execution.mode` — that is `apply`'s job (see `docs/manual/28-apply.md`), and it needs a separate, explicit invocation to run at all.

For each group, `plan`:

1. Fetches the same load (`ℓ_d/L_s/u_s`) `show-load` does, and evaluates the same section 6 gate.
2. If the gate says `NO ACTION`, stops there for that group — no solver runs, nothing more to show.
3. If it says `ACT`, solves it with whichever backend `solver.backend` selects (see `docs/internals/91-optimize.md` for the CP-SAT/CBC backends and `docs/internals/90-heuristic.md` for the dependency-free one) to compute a target assignment, orders its moves under the **transient reserve invariant** — while a move is in flight, the disk being migrated exists on *both* its source and target storage at once (the mirror is fully allocated on the target before the old copy is removed from the source), so the target's own snapshot reserve must already hold with that disk's bytes counted in, not only once the source frees them afterwards (`IMPLEMENTATION_PLAN.md` section 8.1 has the full formula, including the generalization to several moves landing on the same storage at once under concurrent execution — see `docs/manual/28-apply.md`) — and checks the whole plan against the payback rule below.

This example uses the same section 14 worked example `show-load`'s manual page does, at the default weights (the two-move plan) and `solver.backend: auto` with neither MILP library installed, so it falls back to the heuristic — the numbers are traceable to that section either way, since every backend reports through the identical objective function:

```
$ pve-storage-drs -c /etc/pve/drs.yaml plan
Group fc-tier1 → ACT: reserve violated on san-a; acting now regardless of the normal drift/imbalance
thresholds -- a capacity shortfall is never delayed by them
  solver: heuristic
  1. db01(102):scsi0      san-a → san-c    1.50 TiB  ~2.2h  Δimbalance -4.53  ℓ/z 1.67
  2. web01(101):scsi1     san-a → san-b    1.00 TiB  ~1.5h  Δimbalance -2.00  ℓ/z 1.00
  after: san-a=3.00  san-b=1.70  san-c=2.70
  spread: 255.4% → 52.7%
```

```
payback: benefit 1.93e+08 load·s vs cost 2.62e+04 load·s → ratio 7.34e+03 (need 10) ✓
```

The solver: line names whichever backend actually produced this plan – heuristic, cpsat or cbc – plus, for a MILP backend, (optimal) or (feasible) (the gap was not proven closed within solver.time_limit_seconds, but a solution was still found). It is not always what solver.backend says: auto tries CP-SAT then CBC before the heuristic, and even an explicitly forced cpsat/cbc falls back to the heuristic (logged as a warning in that case) rather than failing the whole run, if that library is not installed or cannot solve within the time limit – see docs/internals/91-optimize.md.

The after:/spread: lines (and the payback numbers) reflect what the scheduler actually managed to order, not the solver’s target assignment – identical here since both moves scheduled cleanly, but see the deadlock note below for when a plan cannot schedule everything it proposed.

A group the gate does not act on prints one line and stops:

```
Group fc-tier2 → NO ACTION: imbalance 0.0% is below gates.imbalance_threshold (20.0%)
```

Reading a move line

1. db01(102):scsi0 san-a → san-c 1.50 TiB ~2.2h Δimbalance -4.53 l/z 1.67 — the disk (its VM’s name, then its vmid in parentheses, then the device), its current and target storage, its size, the estimated mirror duration (size / migration.bwlimit_bytes_per_sec, plus a +wipe <time> suffix when saferemove on the source storage adds one — see the payback section below), this move’s own effect on the group’s imbalance metric at the moment it was scheduled (negative means imbalance went down, which is the usual case; a move scheduled mainly to resolve a reserve violation or consolidate a VM can show a positive value and still be correct), and l/z — load per TiB, section 7.3’s own “single best indicator of a good migration candidate: high I/O concentrated in a small disk.”

A move whose own duration exceeds migration.max_single_move_duration carries a Δ exceeds migration.max_single_move_duration suffix and always fails the plan (see below) — this is a hard, per-move rule, independent of whether the plan as a whole looks profitable.

A move scheduled first *regardless* of its ratio — because the storage it leaves is currently breaching its snapshot reserve or its configured free_space.soft requirement, the capacity a storage must always keep free, sized to the larger of snapshot_reserve.factor times its largest disk or free_space.soft (IMPLEMENTATION_PLAN.md section 5.3, constraints (C4)/(C5)/5.3.1) — is implied by the ACT: reserve violated on ... header in human output. Safety is not subject to hysteresis (section 13), so this always wins over a purely balance-driven move.

repair (visible in --json, [repair] in human output) marks a move the plan’s own shortfall *depends on*: holding that one disk back on its current storage, in the plan actually executed, would leave the group short by more than it is. This is not the same question as “did this move’s own source violate something” — an *indirect* repair (a move that empties the destination another repair needs) is marked too, even though its own source was clean, and a *redundant* repair (either of two moves alone would fix the violation) is marked on neither, since holding back just one still leaves the other to finish the job. See the payback: line below for what the plan as a whole being exempt actually depends on — a per-move repair: true marker is a report, not itself the trigger.

A Δ line means a deadlock, not a hidden failure. If the target assignment includes a move this run cannot find any transient-feasible order for, plan says so explicitly (IMPLEMENTATION_PLAN.md section 8.3’s “report, never force” path — staging and plan-splitting are not yet implemented, so this is where a genuine cycle or an unavoidable capacity shortfall currently surfaces) rather than silently omitting the move or, worse, telling you a plan is clean when part of it is not achievable. When only *some* moves

deadlock, the after:/spread:/payback: lines report the state reachable by the moves that did schedule, never the fuller picture the undeliverable ones would have produced.

The payback: line

Section 7.3's acceptance test: $\text{benefit} \geq \text{migration.payback_ratio} * \text{cost}$, both sides in load-seconds. benefit is $(\alpha_{\text{spread}} * \Delta E + \delta_{\text{capacity_spread}} * \Delta F + \kappa_{\text{vm_affinity}} * \Delta A) * \text{migration.payback_horizon}$ (section 7.2's own *unweighted* E/F/A, not the solver's already-scaled objective terms — passing those instead would double-apply the weight); ΔA may be negative, and then it reduces the benefit — a balance move that splits a VM pays for that fragmentation out of its other gains. $E_{\text{after}}/F_{\text{after}}/A_{\text{after}}$ are evaluated against what plan's own scheduler actually managed to order, not the solver's aspirational target, so a partially-deadlocked plan is scored on the moves it can really make, not ones it cannot. cost sums every *scheduled* move's $\text{duration_mirror} * (\text{source_load_weight} + \text{target_load_weight}) + \text{duration_wipe} * \text{wipe_load_weight}$, except a disk below $\text{migration.tiny_disk_bytes}$, which costs 0 regardless of duration (section 7.1). A ✓ plan passed; a × one did not. A plan made entirely of moves below $\text{migration.tiny_disk_bytes}$ has $\text{cost} = 0$ and renders as ratio inf (need 10) ✓ — a real, correct verdict, not a formatting bug.

A × plan can fail for two different reasons, reported as two different lines, because they call for different fixes:

- **Economic failure** — the balance benefit does not outweigh the migration cost. Adjust objective's weights, $\text{migration.payback_ratio}$, or accept the plan is not worth running.
- **Hard-duration failure** — some move's own $\text{duration_mirror} + \text{duration_wipe}$ exceeds $\text{migration.max_single_move_duration}$, regardless of whether the plan as a whole is profitable. Fix `saferemove` throughput, $\text{migration.bwlimit_bytes_per_sec}$, or the duration limit itself.

Either, both, or neither can apply to the same plan; plan names exactly which happened rather than one generic “does not pass the payback test” line for both.

A plan that leaves the group with less reserve/free-space shortfall than it found always passes this test, regardless of the ratio shown — the example above happens to pass on merit ($133 \geq 10$), but a plan whose *only* move fixes a snapshot-reserve or `free_space.soft` violation with zero balance benefit (a real, common case: relocating the sole loaded disk in a two-storage group changes which side carries it without reducing spread at all) is accepted too. This is `--json's payback.repair_exempt`, decided on the plan's *outcome* — its executed endpoint's Σr_s strictly below the current assignment's ($\text{payback.reserve_shortfall_bytes_before}/\text{after}$) — not on any one move's own flag, so a plan that moves a disk off a violating storage but leaves the group no less short in total is **not** exempt. Section 13's “the reserve is never traded against balance” applies here exactly as it does to the gates: an operator does not get to decline a capacity emergency fix because it scores poorly against $\text{migration.payback_ratio}$. The hard per-move duration rule is not exempted this way — it is an operational limit, not an economic one, and still blocks the plan, and a move it excludes is also excluded from what `repair_exempt` scores: a repair a hard rule then blocks is correctly not exempt either.

What a × (or a rejected move) does *not* do today: automatically make the plan smaller and retry. Section 7.3 describes re-solving with β and γ doubled, up to three times, converging on the highest-value subset of moves. That loop is not implemented — a failing plan is reported and left for you to review, not silently adjusted. See `docs/internals/96-payback.md`.

What plan does not yet do

- **No pinned block, fragmentation naming, or pinned-load line.** Unlike `IMPLEMENTATION_PLAN.md` section 9.5's own example, `plan/apply` print only the move list and payback verdict — that narration

(including the per-pin → action hint) lives in `pve-storage-drs explain` instead (`docs/manual/29-explain.md`); `show-load` also shows each pin inline, per disk.

- **No automatic re-solve-and-shrink on a failing payback test** (see above) — reported, not fixed for you.
- **The section 7.3 saturation-ceiling defer check only covers the mirroring phase**, not a second, separate check for the *draining* phase a `saferemove wipe` holds a storage in afterward — that needs `schedule.py` to reason about which moves actually overlap in time, which it does not do. The check itself is otherwise active for any storage that configures `saturation_load` (still none in this project’s own dogfooding cluster) — a deferred move is reported separately from a hard-duration-rejected one (`deferred_moves`, not `rejected_moves`) and excluded from `apply` the same way. See `docs/internals/96-payback.md`.
- **plan itself still only reads state.json, never writes it.** Its gate reads real `last_balance` history when a group has one recorded (see `docs/internals/15-state.md`), and a disk/storage cooldown pins or excludes exactly as described below — but `plan` never executes a migration, so it never has anything of its own to record. `apply` is what writes `last_balance` and cooldowns now, after a run that actually executed at least one migration (`docs/manual/28-apply.md`); a group with no `state.json` yet, or one `apply` has never touched, still evaluates as a first run (drift gate skipped).
- **Cooldowns.** A disk moved within `gates.cooldown_per_disk` is pinned and excluded from this run’s solve — `plan` itself does not print pins at all (see above); `show-load/explain` both show it as `[pinned: cooldown: ...]`. A storage that was a migration’s *destination* within `gates.cooldown_per_storage` accepts no new incoming moves from the heuristic. See `docs/internals/15-state.md`, `docs/internals/60-topology.md` and `docs/internals/90-heuristic.md`.
- **No staging, no concurrent scheduling.** Documented as deliberate, not forgotten, in `docs/internals/95-schedule.md`.

`--json` emits `groups[]`, each with `gate` (identical shape to `show-load`’s), `solver_backend/solver_status` (`null/null` when the gate said `NO ACTION`; otherwise `"heuristic"/null`, or `"cpsat"/"cbc"` with `"optimal"/"feasible"` — the same information the human output’s `solver: line` names), `moves[]` (`disk_key`, `vmid`, `vm_name`, `device`, `from_storage`, `to_storage`, `size_bytes`, `imbalance_reduction`, `repair` (the section 7.3 revert-test marker — see “The payback: line” above), `load_per_tib`, `duration_mirror_seconds`, `duration_wipe_seconds`, `cost_load_seconds`, `exceeds_max_duration`), `deadlocked` (a list of disk keys) and `deadlock_message` (`null` if none), `before_spread/after_spread` (the section 6 spread fraction, before the plan and after every scheduled move), `before_capacity_spread/after_capacity_spread` (the section 5.3 (C7) fill-fraction spread the same way, `null` for a group whose mean fill is 0 or that has not solved), `before_objective_total/after_objective_total` (the section 5.4 objective’s full `.total`, `evaluate_assignment()` re-scored at the same true weights `validate_corpus.py`’s cross-backend check uses — REVIEW.md AA-01), `load_error` (`null` unless Prometheus failed for this group), and `payback` — `null` when there is no `GroupLoad` or the gate said `NO ACTION`, otherwise an object with `benefit_load_seconds`, `total_cost_load_seconds`, `ratio`, `aggregate_ok` (the economic test alone, or `true` if exempted), `repair_exempt` (the outcome trigger itself — see above), `reserve_shortfall_bytes_before/_after` (Σr_s on the current assignment and on the plan’s executed endpoint, what `repair_exempt` is decided from), `rejected_moves` (disk keys failing the hard duration rule), `deferred_moves` (disk keys deferred by the section 7.3 saturation guard — empty unless a storage in the group configures `saturation_load`) and `accepted` (`aggregate_ok` and neither list non-empty).

Reading apply

`apply` runs the identical per-group pipeline `plan` prints — the same gate, the same solver, the same scheduler, the same payback test (`plan`’s own manual page, `docs/manual/27-plan.md`, describes all four in detail) — and then, unless it is dry-run, actually issues the moves. `execution.mode` (or `--mode`) picks

what happens next:

Mode	What apply does
dry-run (<i>default</i>)	Prints the same report plan would, plus a would_move outcome per move. Issues zero API calls.
confirm	Executes one move at a time, prompting before each: [y]es/[n]o skip/[a]ll remaining/[q]uit.
auto	Executes unattended, subject to execution.time_windows, max_migrations_per_run, and a bounded automatic re-plan loop — see “Reading auto mode” below. Also the only mode that honours execution.max_concurrent_migrations/max_concurrent_per_storage above their default of 1, running several moves at once — see “Concurrent execution” below.

A confirm run, using the same section 14 fixture plan’s own manual page walks through, with the operator declining the first move and accepting the rest:

```
$ pve-storage-drs -c /etc/pve/drs.yaml --mode confirm apply
Group fc-tier1 → ACT: reserve violated on san-a; acting now regardless of the normal drift/imbalance
thresholds -- a capacity shortfall is never delayed by them
  solver: heuristic
db01(102):scsi0 san-a → san-c  1.50 TiB  [y]es/[n]o skip/[a]ll remaining/[q]uit? n
1. db01(102):scsi0          san-a → san-c    1.50 TiB   ~2.2h   Δimbalance -4.53   l/z 1.67 → skipped:
operator declined
web01(101):scsi1 san-a → san-b  1.00 TiB  [y]es/[n]o skip/[a]ll remaining/[q]uit? a
2. web01(101):scsi1          san-a → san-b    1.00 TiB   ~1.5h   Δimbalance -2.00   l/z 1.00 → moved:
task UPID:... completed OK, source released
after: san-a=5.50 san-b=1.70 san-c=0.20
spread: 255.4% → 214.9%
payback: benefit 3.02e+07 load·s vs cost 1.05e+04 load·s → ratio 2.88e+03 (need 10) ✓
```

Everything above the → status: detail suffix on each move line is identical to plan’s own output — the “after”/“spread”/“payback” lines still describe the plan as scheduled, not adjusted for what a declined move would have changed, since (as with a partial deadlock, see plan’s manual page) reporting what was actually asked for is more honest than recomputing a hypothetical.

The [y]es/[n]o skip/[a]ll remaining/[q]uit prompt

Only in confirm mode, once per move, before it is issued — never for a move dry-run would only report. [a]ll remaining stops prompting for the rest of *this run* (every group, not just the current one); [q]uit stops the whole run immediately, including any group not yet even planned. An answer that is not one of the four is rejected right there (“please answer y, n, a or q”) and asked again — it never reaches execute.py at all, so a typo can never be mistaken for one of the four real decisions.

Reading auto mode

auto behaves like [a]ll remaining chosen up front, unprompted, plus three things dry-run/confirm do not do at all (IMPLEMENTATION_PLAN.md section 9.1/9.2):

- **execution.time_windows.** If any are configured, a move is refused (reported skipped, “outside execution.time_windows”) unless the current moment — in the **local time of the host running pve-storage-drs**, not UTC — falls inside one of them, and before starting a move auto also refuses it if its estimated duration (mirror plus any saferemove wipe) would not finish before the window closes. No windows configured at all means no restriction. Either way, a move already in progress is never aborted at window close — the check only ever runs *before* issuing the next one.

- **execution.max_migrations_per_run.** A ceiling on how many moves the whole invocation executes, shared across every group it visits (not reset per group) — the remainder waits for the next scheduled run.
- **The re-plan loop.** A `replan_needed` mismatch does not end an auto run the way it ends a dry-run/confirm one: `apply` re-invokes the whole pipeline (gates, load model, solver, payback, ordering) from freshly observed cluster state and tries again, up to `execution.max_replans_per_run` times. The gate may well conclude no further action is needed on the re-plan — a normal, quiet outcome, not a failure. Exceeding the cap ends the run with a message naming the setting, and the run still exits 0: too many outside changes (VMs moved, disks created, another tool filling a target) to plan against is a reason to try again later, not a failure of the tool, and the next run starts from freshly observed state. A re-plan that cannot be computed because Prometheus cannot be read is different — see “When something cannot be read” below. Cross-referencing the report’s `outcomes[]` shows the whole story for a re-planned group: the mismatch that triggered each re-plan, and whatever the next attempt then did.

Concurrent execution

Configuring `execution.max_concurrent_migrations/max_concurrent_per_storage` above their default of 1 runs several moves at once in auto mode (dry-run/confirm always run strictly sequentially, regardless of these settings — concurrency only makes sense for unattended operation). The **transient reserve invariant** — the rule (docs/manual/27-plan.md) that a storage’s snapshot reserve must hold even while a disk migrating onto it exists on both source and target at once, not only once the move finishes — generalizes to a whole in-flight set of moves landing on the same storage at the same time (IMPLEMENTATION_PLAN.md section 8.1’s own formula for it), checked live before every launch exactly like the sequential executor’s own pre-flight re-check; `max_concurrent_per_storage` counts a storage as occupied whether a move touches it as source or target.

Launch order stays strictly FIFO. The scheduler’s own queue (the same one a sequential run would follow, one move at a time) is never reordered to keep every concurrency slot busy: if the next queued move cannot launch yet — its VM is locked, or a per-storage cap is already reached — `apply` waits for it rather than skipping ahead to a later move that could launch immediately. Several moves genuinely run at once once launched (nothing blocks waiting for one move’s own completion or lock to affect any other), but which move launches *next* is always decided in the order `plan` would have shown it. This can under-deliver on throughput in a mixed queue, but never launches a move out of the order the payback-scored plan itself computed.

A failure (or a budget/deadline limit) stops the run from launching anything further, exactly as in the sequential case, but does not abandon moves already in flight: `apply` keeps polling them to their own natural conclusion (moved, failed, or draining) before returning, so nothing already running is left unreported. Section 7.3’s saturation guard is enforced at planning time under concurrency exactly as under the sequential executor (a flagged move never reaches either executor); what neither executor does is re-check the ceiling *during* execution against the live in-flight set (docs/manual/30-safety-and-status.md’s own note on that narrower gap applies here too).

What “done” means for one move, and why it can take a while

A move is not finished when the `move_disk` task itself succeeds. IMPLEMENTATION_PLAN.md section 9.3 requires all three, together: the task reports `exitstatus: OK`, the source volume is gone from the source storage’s own content listing, and the VM’s config lock is clear again. A storage with `saferemove` enabled zeroes the old volume afterwards at `saferemove_throughput` (default 10 MiB/s; a negative value in `storage.cfg` is a rate too, see [verify-storages](#)) — for a large disk, that can run for hours after the task itself is long done. While it runs, `apply` reports the move as draining, not moved and not failed:

it is a normal, expected state, bounded by `execution.source_release.timeout` (default 48h), not an error. A draining move still counts as “executed” for `state.json`’s bookkeeping below — the mirror itself completed; only the source’s own cleanup is still in flight. For the *rest of this same run*, that storage is excluded as both a source and a target for any later move: it holds a storage-level lock for as long as the wipe runs, so a subsequent `move_disk` touching it would either queue behind a wipe that can take days or fail outright. A move skipped this way is reported plainly (skipped, naming the draining storage) rather than attempted into that lock.

Before *every* move, `apply` re-checks the live cluster rather than trusting the plan: the VM may have moved node, the disk may no longer be on the expected source, a snapshot may have appeared, it may have been tagged for exclusion, or the target storage’s free space may no longer satisfy the transient invariant (the target must still hold every disk on it, plus the disk being moved, plus the snapshot reserve, while both copies of the moving disk exist). That last re-check reads the target’s volume listing afresh and counts every volume at its *provisioned* size, exactly as planning did — not the pool’s own allocated figure, which on a thin-provisioned pool is much lower and would let a move through onto a pool that other provisioning has since filled. A mismatch — the VM moved, a snapshot appeared, the target no longer has room — stops the group’s run with `replan_needed`. In dry-run/confirm, that is the end of it — the operator re-runs `apply` by hand once ready; see “Reading auto mode” below for what auto itself does about it automatically.

If a check cannot be *made* at all — the PVE API errors while re-reading the VM or the target, or the target lists a volume with no size — the move is refused and the whole run fails; see “When something cannot be read”.

A VM config lock (backup, snapshot, migrate, or any other value — this set is never whitelisted, see `docs/internals/92-execute.md`) makes `apply` wait, not fail, up to `execution.locks.wait_timeout_seconds`. What happens on that timeout depends on `execution.locks.on_timeout`: `skip` (the default) reports the move skipped and continues with the rest of the plan; `abort` reports it failed and stops the *whole run* unconditionally — this one case ignores `execution.abort_on_failure` entirely, because the manual’s own words for `on_timeout: abort` are “abort the run,” a distinct, stronger promise than the general “stop after any failed move” policy a plain task failure still goes through.

Failure and `abort_on_failure`

A failed `move_disk` task is reported failed with its detail; `apply` then lists any volume left behind on the target that is not referenced in the VM’s config (`IMPLEMENTATION_PLAN.md` section 9.4) — **never deleted, only reported**, exactly like every other orphan this project surfaces. With `execution.abort_on_failure: true` (the default), the whole run stops there; with it `false`, `apply` continues with the plan’s remaining moves. The lock-timeout-abort case above is the one exception that always stops the run regardless of this setting.

When something cannot be read

A mismatch between the plan and the cluster is something `apply` can plan around; a *failure to look* is not. These end the whole run at once — no later group is planned or executed — and `apply` exits 1:

- the PVE API errors while `apply` re-reads a VM or the target storage just before a move (or the target lists a volume with no size);
- Prometheus cannot be read while computing a group’s load — in the first plan, or in an auto re-plan. (A re-plan that cannot be computed is a failed run, not “no further action needed”).

Moves that already completed stay recorded in `state.json` and in the report, and the report says what could not be read. Planning around data it could not observe would be a worse plan, not a better one, so

the tool does not try: fix the connection and run it again. plan and explain, which change nothing, still report every group they can — and also exit 1 when any group’s load could not be computed.

state.json: what a real run actually changes

apply is the one command that takes state.json’s advisory lock (docs/internals/15-state.md) — checked first, before it ever touches PVE or Prometheus, so a second concurrent apply (any mode, including dry-run) exits 0 immediately and quietly rather than racing the first one. After the run, for every group where at least one move actually executed (moved or draining — both mean the mirror itself completed):

- last_balance.load_vector is replaced with that group’s load as measured for this run, so the next run’s drift gate compares against a real balance instead of treating every run as the first one.
- Every executed disk gets a fresh gates.cooldown_per_disk timestamp at its **new** location.
- **Both** of the move’s storages — source and destination — get a fresh gates.cooldown_per_storage timestamp: section 6 says a storage “involved in a migration” accepts no new incoming moves, and section 9.3’s sizing rule for this cooldown is specifically about protecting a *source* still draining a saferemove wipe, so recording the destination alone could never deliver on it. What the cooldown actually *excludes* on the next run stays destination-only (the heuristic/MILP backends refuse a cooldown storage as an incoming-move target, never as a source a disk may still leave) — recording both endpoints only widens which storages carry a timestamp, not what that timestamp blocks.

A group that never executes anything (the gate said NO ACTION, or a load_error/replan_needed stopped it before any move ran) leaves state.json untouched for that group.

Crash and two-instance recovery

Before planning anything, every apply run (including dry-run) checks for a move_disk left running by a crashed previous run or a second, concurrently-running instance: state.json’s own inflight_upids are re-checked, and every node’s task list is scanned for a still-running qmmove task from this tool’s own configured user. Any vmid this finds is excluded from this run’s planning exactly like a manually-configured exclude.vuids entry — it shows up pinned in the report with the reason “excluded by config (exclude.vuids)”, and a log line at WARNING explains which check actually found it. It is never force-cancelled, and this run never assumes it is safe to touch that VM’s disk again.

During execution, state.json’s inflight_upids is written **before** each move_disk is issued and cleared once that task itself has finished — synchronously, so a crash (killed, OOM, host reboot) partway through a move leaves a trace the *next* run’s own startup check will find, rather than only ever being updated at the very end of a whole apply run. See docs/internals/93-crashrecovery.md for the full mechanism.

--json

Identical to plan’s own groups[] shape (see docs/manual/27-plan.md), with one addition per group: “execution”, either null (never reached — a load_error, NO ACTION, or a group the run never got to because an earlier one stopped it, e.g. an operator’s [quit] or an object with stopped_early, stop_reason (null unless the run stopped for this group specifically), aborted (true when this group’s execution hit one of the failures above and the run ended because of it), replans_exhausted (true when execution.max_replans_per_run ran out — the run bailed out and exits 0) and outcomes[] — one entry per move actually attempted, each with disk_key, vm_name, from_storage, to_storage, status (would_move/moved/skipped/failed/draining/replan_needed), detail, upid (null unless move_disk was actually issued for this move — always null for would_move/replan_needed, and for skipped except a lock timeout that happened after issuing nothing yet, so still null there too; a failed outcome carries one only when the task itself ran and failed, not when a lock timeout aborted before it started) and orphaned_volumes (only ever non-empty after a failed outcome).

Reading explain

`explain` runs the identical `gate/solve/schedule/payback` pipeline `plan` does (`docs/manual/27-plan.md`) — same load, same gate, same solver, same scheduler, same payback test, byte-for-byte the same numbers — and prints everything `plan` already shows, plus the “why” `plan` itself never does: the measured load every one of those numbers derives from, which disks are pinned and the exact reason, which VMs a pin leaves spread across more than one storage, the section 5.4 objective broken into its six terms, and whether the pinned load is large enough that the residual imbalance is structural rather than a planning shortfall. `-v` additionally names exactly which Prometheus query produced all of it. It never changes anything, in any `execution.mode` — exactly like `plan`.

This example extends the section 14 worked example `plan`’s own manual page uses with one more VM, 106 (`archive01`): its `scsi0` carries two real snapshots, so it is pinned (section 3.7), while its `scsi1` is an ordinary movable disk on a different storage — the “cannot fully consolidate” case section 3.6 exists to name:

```
$ pve-storage-drs -c /etc/pve/drs.yaml explain
Group fc-tier1 → ACT: imbalance 255% exceeds gates.imbalance_threshold (20%)
  solver: heuristic
  1. db01(102):scsi0      san-a → san-b      1.50 TiB  ~2.2h  Δimbalance -4.00  l/z 1.67
  2. web01(101):scsi0     san-a → san-c      2.00 TiB  ~2.9h  Δimbalance -3.40  l/z 1.50
  3. mail01(103):scsi0    san-b → san-a     512.00 GiB ~43.7m  Δimbalance -0.80  l/z 0.80
  4. cache01(105):scsi0   san-c → san-a     512.00 GiB ~43.7m  Δimbalance -0.40  l/z 0.40
  after: san-a=2.50  san-b=2.90  san-c=3.00
  spread: 257.1% → 17.9%
  payback: benefit 2.52e+08 load·s vs cost 4.72e+04 load·s → ratio 5.34e+03 (need 10) ✓
  objective: imbalance 0.6 + moves 1 + bytes 0.225 + fragmentation 1.43 + spread 0.25 + reserve 0 = 3.5
  measured load:
  san-a  provisioned 5.50 TiB/8.00 TiB  L=7.40 u=7.40  Δ reserve short by 1.50 TiB  (largest disk 2.00 TiB,
  requires 4.00 TiB free)
    web01(101):scsi0      2.00 TiB  raw    l 3.00
    web01(101):scsi1      1.00 TiB  raw    l 1.00
    db01(102):scsi0       1.50 TiB  raw    l 2.50
    archive01(106):scsi0   1.00 TiB  raw    l 0.90 [pinned: snapshots present (2)]
  san-b  provisioned 2.00 TiB/8.00 TiB  L=0.80 u=0.80  reserve OK  (largest disk 1.00 TiB, requires 2.00 TiB
  free)
    mail01(103):scsi0     512.00 GiB  raw    l 0.40
    app01(104):scsi0       1.00 TiB  raw    l 0.30
    archive01(106):scsi1   512.00 GiB  raw    l 0.10
  san-c  provisioned 512.00 GiB/8.00 TiB  L=0.20 u=0.20  reserve OK  (largest disk 512.00 GiB, requires 1.00
  TiB free)
    cache01(105):scsi0    512.00 GiB  raw    l 0.20
  pinned (not movable this run):
    archive01(106):scsi0   1.00 TiB  on san-a  l 0.90  l/z 0.90  -- snapshots present (2) → clear
    snapshots to unblock
  cannot fully consolidate:
    106 (archive01)  scsi0: snapshots present (2)
  pinned load 0.90 of 8.40 (10.7%, warn at 25%);  best achievable spread given pins: 17.9%
```

Everything up to and including the `payback`: line is identical to `plan`’s own output for the same input — see `docs/manual/27-plan.md` for how to read the move lines and the `payback` verdict. What follows is new.

The objective: line

The section 5.4 objective the solver actually minimized, broken into its six terms rather than only the total — imbalance (alpha times the spread metric), moves (beta times the move count), bytes (gamma times TiB moved), fragmentation (kappa times each VM’s I/O-weighted extra storage count beyond one — section

5.4's `w_v`, so a VM doing several times the group's average I/O costs correspondingly more to split), spread (delta times the section 5.3 (C7) data-spread deviation), and reserve (the penalty for any remaining (C5) shortfall, zero on a plan that resolves or never had one). This is the same heuristic. `ObjectiveBreakdown` the solver itself compares candidate assignments with — the reason that class keeps the six terms apart instead of collapsing to only `.total` in the first place. Only printed when the gate said ACT; a NO ACTION group solved nothing this run, so there is no objective to show.

no moves made: ... / closest alternative: ...

Only printed when the gate said ACT *and* the solver's own optimum still moves nothing — every disk eligible, nothing pinned, yet the objective is lowest at the current assignment. This is a real, expected outcome, not a bug: it happens when the only move (or moves) that would meaningfully improve the spread costs more, in moves/bytes/fragmentation, than the imbalance it fixes is worth under the configured weights. Without this section, that case is indistinguishable from “the solver didn't try” — plan's one-line verdict says ACT either way.

`explain` finds the single-disk move closest to being worth taking — every movable disk against every other storage in the group, evaluated with the same objective the solver itself minimizes (section 5.3's one-move neighbourhood) — and shows the term-by-term arithmetic that rejected it:

```
objective: imbalance 0.576 + moves 0 + bytes 0 + fragmentation 0 + spread 0 + reserve 0 = 0.576
no moves made: the objective is lowest at the current assignment
closest alternative: vm110(110):scsil VM-krbd → VM
imbalance 0.576→0.0426, moves 0→0.25, bytes 0→0.00732, fragmentation 0→0.5, spread 0→0, reserve 0→0
total 0.576 → 0.8 (worse by 0.223 -- rejected)
```

Read left to right: moving `vm110(110):scsil` from VM-krbd to VM would cut the imbalance term from 0.576 to 0.043 — a real improvement — but 110 is a two-disk VM with its other disk staying put, so the move splits it across two storages, and `objective.kappa_vm_affinity`'s fragmentation penalty ($0 \rightarrow 0.5$) for that outweighs the gain. If this is a tradeoff you want to accept rather than reject, lowering `objective.kappa_vm_affinity` (or `objective.beta_move_count/gamma_move_bytes_per_tib`, whichever term dominates the rejection) is the knob — not a reason to suspect the solver.

Omitted entirely when there is nothing to compare against (every disk pinned, or a one-storage group) or when the plan actually moved something.

measured load:

The section 4 input every number above derives from — identical to `show-load`'s own per-storage, per-disk report (`docs/manual/25-show-load-and-verify-storages.md`): each storage's used/capacity, `L_s/u_s`, reserve status, and every disk on it with its size, format, measured `l_d` when one was fetched, and `[pinned: ...]` when section 5.3 (C2) excludes it. Always present, printed even for a NO ACTION group and even when no migration was possible at all — it is the data the rest of `explain`'s narrative is *about*, not part of the plan itself.

-v: the data source: line

One extra line, printed once for the whole run rather than once per group (every group here was computed against the identical selector and window): the exact section 3.4 node-scoping filter this run's queries carried ((no node-scoping filter) if none applied), and the `window.lookback/quantile/metrics.rate_window/metrics.step` settings the load above was computed from — see `docs/manual/10-configuration.md` for what each controls and `IMPLEMENTATION_PLAN.md` section 3.4 for the filter itself. This is the one piece of `explain`'s output that is about *how* the data was fetched rather than what it is, which is why it is the one thing here gated behind `-v` instead of always shown. By default this is the

node-list alternation from this cluster’s own node names; with `metrics.extra_selector` set, it is that instead:

```
$ pve-storage-drs -c /etc/pve/drs.yaml -v explain
data source: {nodename=~"pve01|pve02|pve03"} window 1.0d lookback, quantile 0.95, rate_window 5.0m, step
5.0m

Group fc-tier1 → ACT: imbalance 255% exceeds gates.imbalance_threshold (20%)
...
```

(With `metrics.extra_selector: 'cluster="mycluster"'`, the same line instead reads `data source: {cluster="mycluster"}.`)

pinned (not movable this run):

Every disk section 5.3 (C2) excludes from this run’s solve, in the same `name(vmid):device` order `show-load` lists disks, with its size, current storage, measured load (l) and l/z ratio when one is available, and its exact reason — a real snapshot or an unreferenced companion volume (section 3.7), an unapplied pending config change (section 3.8), a config exclusion, a still-running `gates.cooldown_per_disk`, or a VM config lock. Identical text to what `show-load` already prints per disk as `[pinned: ...]`; gathered here into one block instead of interleaved with movable disks, because pins are `explain`’s subject, not an aside.

A pin with something to actually act on or wait for gets a trailing `<hint>` — “clear snapshots to unblock” for a real snapshot, “remove the stale reference to unblock” for an orphaned volume, “apply the pending change (reboot the VM) or revert it, then re-check” for a pending config change, “re-check next run” for a cooldown, “re-check next run once the lock releases” for a VM lock. A standing policy exclusion (`exclude.*`, or a deliberately skipped unusedN disk) carries no hint — that pin is not something to unblock, it is a choice already made in the config.

cannot fully consolidate:

Names a VM whose disks — after everything this run’s plan actually managed to schedule — still sit on more than one storage, *because* at least one of them is pinned. A VM legitimately spread across storages by an ordinary, unpinned plan is not reported here; that is the plan working as intended, not a blocker. This is section 3.6’s own example almost verbatim: “evacuating a storage completely is ... not achievable [when] it is because of a section 3.7 snapshot or an explicit exclusion, never because of a device type” — `explain` names which one, and on which device, rather than a plan quietly leaving a stray volume behind.

pinned load ... ; best achievable spread given pins: ...

`report.warn_pinned_load_fraction` (default 0.25) compared against this group’s pinned load as a fraction of its total measured load. Below the threshold, as in the example above, this line is purely informative. Above it, a second, Δ -prefixed line follows: the residual imbalance may be structural — enough load is pinned that no achievable placement of the *movable* disks alone gets the group much closer to balanced — rather than a sign the solver or the weights need attention. `best achievable spread given pins` is exactly the after: `spread` line above: every solve (heuristic or MILP) already fixes every pinned disk at its current storage and optimizes only the movable ones (section 5.1’s $x_{\{d, \sigma_0(d)\}} = 1$ for d outside D^{mov}), so what the plan actually reached is the best that pinned set permits — clearing the pins named above is the only way to do better, not a bigger objective weight or a longer `solver.time_limit_seconds`. Omitted entirely for a group with no measured load at all (an idle group, or one `explain` could not get a load for).

A NO ACTION group still gets the measured-load/pinned/fragmentation/ pinned-load sections above — they describe the state of the group’s disks, not the plan, so there is no reason to withhold them just because the gate found nothing to balance this run.

--json emits everything plan --json does (docs/manual/27-plan.md's own field list) plus objective (the six terms above, null when the gate said NO ACTION), rejected_alternative (null unless the "no moves made" case above applies, otherwise disk_key, vmid, vm_name, device, from_storage, to_storage, baseline and objective — each the same six-term breakdown objective above serializes, for the current assignment and the candidate respectively — and worse_by, the difference between the two totals), storages/disks (the measured-load section above, identical shape to show-load --json's own fields of the same name), pinned_disks (disk_key, vmid, vm_name, device, current_storage, size_bytes, load, load_per_tib, reason, action_hint — null for a standing policy exclusion, same rule as the human report's → line), fragmentation (a list of {vmid, vm_name, blockers}, each blocker an object with device, disk_key, reason), and pinned_load (null for an idle group, otherwise pinned_load, total_load, fraction and warn_fraction). Unlike the human report, JSON has no notion of -v: a top-level query object (node_selector, window_lookback_seconds, quantile, rate_window_seconds, step_seconds) is always present, once per response rather than once per group, the JSON counterpart of the -v data source: line above.

Safety properties, exit codes, and what this build actually does

What is safe, unconditionally

These hold regardless of execution.mode, and are not configurable away (.agents/domain-invariants.md):

- **Dry-run is the default.** A missing or unparseable execution.mode means dry-run, never automatic execution.
- **The snapshot reserve is never traded for balance, and neither is the free space you configure.** pve-storage-drs will leave a group imbalanced rather than breach snapshot_reserve.factor/free_space.soft on any storage, and that reserve holds *during* a migration, not merely before and after it — down to free_space.hard, never below it.
- **Nothing is ever deleted automatically.** An orphaned volume left by a failed move is reported, never removed. The operator deletes.
- **An explicitly named configuration that cannot be read or fails validation is a hard failure,** never a silent fallback to /etc/pve/drs.yaml.

Exit codes

Code	Meaning
0	Success. A run that stopped at a gate, found nothing worth moving, or (for verify-metrics) found no error-level finding.
1	The run failed: invalid configuration, an unreachable Prometheus or PVE API (including one that fails mid-run, for any group, in the first plan or in a re-plan), a failed move, a failed check, or a command not yet implemented in this build (see below). A run that <i>bails out</i> because execution.max_replans_per_run was exhausted is not a failure and exits 0.
2	Usage error on the command line.

What this build actually implements

IMPLEMENTATION_PLAN.md section 12 lays out thirteen phases, all thirteen now implemented, including phase 10 (collect-testdata, --replay and the real-cluster test corpus of section 16), phase 11 (the section 2.3 logging policy — see 35-logging.md; the status file a monitoring check reads is 36-monitoring.md), phase 12 (the capacity-spread objective and gate, section 5.3 (C7)/5.4's 6 term) and phase 13 (per-storage free-space requirements, section 5.3.1). Dogfooding against a production cluster continues, and REVIEW.md

tracks findings from that to resolution — being honest about exactly where a still-evolving tool stands matters more than a document that reads as finished and closed:

Command	Status
--version, --help, --manual	Implemented.
verify-metrics	Implemented: all six IMPLEMENTATION_PLAN.md section 3.3 checks, human and --json output.
show-load	Implemented: every storage in every group, its disks (all buses), sizes, reserve status ((C4)/(C5)), per-disk/per-storage I/O load (l_d/L_s/u_s, section 4), and a section 6 act/no-act verdict per group with its reasoning, pinned and low-coverage disks flagged with their reason (including a disk within gates.cooldown_per_disk, once state.json has recorded one), human and --json output. A Prometheus outage degrades this one group's load (and its gate verdict) to "unavailable" rather than failing the whole command — sizes and reserve status are unaffected. The gate verdict now reads real drift history from state.json when a group has one recorded; a group with none yet still evaluates as a first run (reserve override or imbalance only, never drift-suppressed) — see docs/internals/15-state.md and docs/internals/80-gates.md.
verify-storages	Implemented: saferemove and the implied wipe time per storage, the resolved free_space.soft/.hard bytes per storage (section 5.3.1), warning when gates.cooldown_per_storage or migration.max_single_move_duration is shorter than the implied wipe time.
plan	Implemented: per group, evaluates the section 6 gate (reading real drift history from state.json when a group has one recorded — same as show-load, see docs/internals/15-state.md), and on ACT solves it with solver.backend (auto tries CP-SAT, then CBC, then the dependency-free heuristic — see docs/internals/91-optimize.md; both MILP backends and the heuristic exclude, as a migration target, any storage within gates.cooldown_per_storage, except when repairing a live reserve violation, which is never deferred for a cooldown, in the heuristic only — see docs/internals/91-optimize.md's own note on that gap), orders its moves under the section 8 transient reserve invariant, and checks the whole plan against section 7's payback (cost/benefit) test, including the section 7.3 saturation-ceiling guard (mirroring phase only) for any storage that configures saturation_load (still none in this project's own dogfooding cluster) — a plan whose executed endpoint reduces the group's reserve/free-space shortfall (section 5.3.1's free_space.soft) is exempt from the economic half of that test but not from the hard per-move duration rule or the saturation guard. Human and --json output report which backend actually solved each group. No automatic re-solve-and-shrink on a failing payback test — a plan that fails is reported, not silently adjusted (section 7.3's 3-retry loop is not implemented). See docs/manual/27-plan.md and docs/internals/90-heuristic.md/91-optimize.md/95-schedule.md/96-payback.md.

Command	Status
explain	Implemented: runs the identical per-group gate/solve/schedule/payback pipeline <code>plan</code> does (<code>_plan_group()</code>), then narrates what <code>plan</code> 's own output does not print. Lists every disk pinned this run with its exact reason (section 3.6/3.7 — snapshots, an exclusion, a lock, or a still-running gates <code>.cooldown_per_disk</code>), names any VM a pin leaves with disks spread across more than one storage (“cannot fully consolidate”), and breaks the section 5.4 objective down into its six terms (ObjectiveBreakdown's own reason for keeping them separate rather than only a total) when the gate acted. Also reports pinned load as a fraction of the group's total against <code>report.warn_pinned_load_fraction</code> , flagging when the residual imbalance is likely structural (too much load pinned to fix by moving anything) rather than a planning shortfall — alongside the best achievable spread the plan actually reached given those pins. Human and <code>--json</code> output, same shape as <code>plan</code> 's with these fields added. See <code>docs/internals/40-cli-and-logging.md</code> and <code>docs/internals/90-heuristic.md</code> .
apply	Implemented for all three <code>execution.mode</code> values. Before planning anything, checks for a <code>move_disk</code> left running by a crashed previous run or a second concurrent instance (section 13) and excludes its <code>vmid</code> from this run; runs the identical per-group gate/solve/schedule/payback pipeline <code>plan</code> prints, then executes it: pre-flight re-checks each move against the live cluster immediately before issuing it (including exclusion tags/ <code>vmids</code>), waits out a VM config lock rather than failing, and does not consider a move done until the task succeeds <i>and</i> the source volume is gone <i>and</i> the VM's lock is clear — a <code>saferemove</code> wipe can hold those apart for hours (<code>draining</code> , excluded as both source and target for the rest of that run). <code>state.json</code> 's <code>inflight_upids</code> is written before each <code>move_disk</code> and cleared once its task finishes, so a crash mid-move leaves a trace the next run's own startup check finds. <code>auto</code> additionally honours <code>execution.time_windows</code> (local host time; refuses a move that cannot finish before the window closes) and <code>execution.max_migrations_per_run</code> (a shared budget across every group the run visits), and re-invokes the whole pipeline from freshly observed state on a <code>replan_needed</code> mismatch, bounded by <code>execution.max_replans_per_run</code> (section 9.2's re-plan protocol). <code>execution.max_concurrent_migrations/max_concurrent_per_storage</code> above their default of 1 are honoured in <code>auto</code> mode: section 8.1's generalized transient invariant and section 9.2's “poll all in-flight UPIDs” loop, strictly FIFO (never reordering the scheduler's own queue to keep every slot busy) — <code>dry-run/confirm</code> always run strictly sequentially regardless. See <code>docs/manual/28-apply.md</code> , <code>docs/internals/92-execute.md</code> and <code>docs/internals/93-crashrecovery.md</code> .

Command	Status
collect-testdata, and the global --replay	Implemented (IMPLEMENTATION_PLAN.md section 16, phase 10). collect-testdata captures everything the engine can read from the PVE API and Prometheus — the superset of every forecaster's data range, not just the configured one — replaces every identifier with a pseudonym keyed to a salt that never leaves the host (support.salt_path), and writes a bundle (directory + deterministic .tar.gz) you can read before deciding whether to send it to the author as test data; --estimate prints the query/sample-point estimate and refuses before fetching above support.max_series_points. --replay BUNDLE then runs verify-metrics, verify-storages, show-load, plan and explain against such a bundle with no network access at all, using the bundle's own config.yaml unless -c overrides it; apply and any --mode above dry-run are a usage error under --replay (exit 2), and so is collect-testdata itself, which needs a live cluster. See docs/manual/26-collect-testdata-and-replay.md and tests/corpus/README.md, which documents what a bundle does and does not hide and the terms on which one is accepted into the project.

Configuration loading and validation (this whole manual's [10-configuration.md](#)) is complete and exercised by every command: a config error is reported and the run exits 1 before `main()` ever dispatches to a command's own handler, so validating a configuration file does not require the rest of the engine to be involved at all.

Every command in the table above now has a real handler, but the dispatch mechanism that reports one honestly if it did not would still print a message naming the plan section it belongs to, and exit 1 — docs/internals/40-cli-and-logging.md describes it as the extension point for the next new command this codebase adds, not dead code kept around for its own sake.

Optional dependencies

pve-storage-drs runs, plans and executes — every `execution.mode` (dry-run/confirm/auto) included — with only requests, ruamel.yaml and jsonschema installed. Two dependencies are still optional and are imported only where they are used, never at module level:

- **ortools** (`pip install proxmox-storage-drs[solver]`, **not** packaged for Debian — CP-SAT has no Debian package at all) — the preferred MILP backend (`solver.backend: cpsat`, or auto's first choice) when installed by hand outside the Debian path.
- **statsmodels** (Debian Suggests) — needed only for `forecast.model: holt_winters`. Without it, that model logs a warning and falls back to quantile automatically rather than failing.

pulp + coinor-cbc are a Debian Depends, not a Recommends: on a Debian install a real MILP solver — CBC through pulp — is always present, and `solver.backend: auto` (the default) uses it unless CP-SAT is also available. The dependency-free heuristic of IMPLEMENTATION_PLAN.md section 5.5 is still a genuine fallback, not dead code — it runs when a non-Debian install has no solver extra, when a solve fails or times out, or when `solver.backend: heuristic` is configured explicitly — but it is no longer what a plain `apt install pve-storage-drs` lands on by default. See docs/internals/91-optimize.md.

Logging: what lands where, and what an unattended run records

Two rules cover almost everything:

- **The report goes to stdout, the log goes to stderr.** Always, in every mode. `pve-storage-drs --json plan > plan.json` is safe: nothing is ever interleaved into that file.
- **A run in which nothing went wrong logs nothing.** By default only warnings and errors are logged, so a healthy plan, explain or show-load prints its report and not one line beside it.

Verbosity

	Level	What you get
<code>--quiet</code> (default)	errors warnings	The run failed, nothing else Something degraded but the run continued
<code>-v</code>	info	This run's decision trail (below)
<code>--vv</code>	debug	Per-query detail, plus urllib3/proxmoxer library logs

`--log-level error|warning|info|debug` states a level outright and wins over both `-v` and `--quiet` — for automation that would rather name a level than count vs. The one exception is the mandatory floor below (“Unattended runs log this without being asked”): `--log-level warning/error` on a `confirm/auto` apply run is raised back up to `info` rather than silently discarding the audit trail. `--quiet` is the only flag that discards it.

The decision trail

`-v` logs, per group, the reasoning behind what the report shows: the measured load (`load_digest`), the gate's computed drift/imbalance together with the thresholds they were compared against (`gate_decision`), the chosen plan with its solver backend and the six objective terms (`plan_selected`), section 7's payback arithmetic and the ratio it needed (`payback_verdict`), and every migration issued with its PVE task UPID (`move_started` / `move_finished`). A run opens with `run_started` and closes with `run_summary` — moves issued, succeeded, failed, bytes moved, wall time and exit code.

```
$ pve-storage-drs -v plan
run started: plan (dry-run)
group fc-tier1: load 7.400 across 6 disks
group fc-tier1: ACT -- imbalance 255.0% meets or exceeds gates.imbalance_threshold (20.0%)
group fc-tier1: cpsat plan, 2 move(s)
group fc-tier1: payback ratio 151 (need 10) -> accepted
run finished: plan exit=0
```

Unattended runs log this without being asked

apply in `confirm` or `auto` mode emits the whole decision trail above **whether or not `-v` was given**. In those modes the log is the only record of what happened to your cluster, and a record you only get if you remembered a flag is not a record. Read-only commands and `apply --mode dry-run` change nothing, so they stay at the ordinary default.

`--quiet` still wins, including over this. It is the one option that will leave you with an `auto` run that migrated data and no account of what it moved or why — which is why a timer unit should not pass it.

Text or JSON

`--log-format auto` (the default) writes **human-readable text when stderr is a terminal and one JSON object per line everywhere else** — a pipe, a redirect, or `journald` under `systemd`. So the same command reads well by hand and parses correctly under a timer, with nothing to configure:

```
$ pve-storage-drs -v plan 2>&1 >/dev/null | head -1
run started: plan (dry-run)

$ pve-storage-drs -v plan 2>log.json >/dev/null ; head -c 120 log.json
{"command": "plan", "event": "run_started", "level": "INFO", "logger": ...
```

--log-format text and --log-format json force one or the other in either direction.

Every JSON record carries an event, which is the supported way to filter:

```
journalctl -u pve-storage-drs.service -o cat | jq -r 'select(.event=="move_started") | .upid'
```

Under systemd

Point StandardError= at the journal and let the defaults do the rest: the non-TTY stream selects JSON on its own, and auto mode raises its own verbosity to log the audit trail. Do not add --quiet; do not add -v (it is only needed if you also want the trail from a dry-run timer).

Monitoring: the status file

A timer-driven pve-storage-drs apply --mode auto that fails quietly is worse than one that fails loudly, and the journal is not a monitoring system. Set `monitoring.status_file` and apply leaves a small report of its last run in the format of the `check_statusfile` plugin from Debian's monitoring-plugins-contrib package (`/usr/lib/nagios/plugins/check_statusfile`). Nagios, Icinga, or anything else that runs a Nagios plugin can then watch it with nothing of ours installed on the monitoring side:

```
/usr/lib/nagios/plugins/check_statusfile /var/lib/pve-storage-drs/status
/usr/lib/nagios/plugins/check_statusfile -a 1h /var/lib/pve-storage-drs/status
```

-a is the maximum age of the file (s, m, h or d; default 26h) — see “Freshness” below.

What is in the file

A worked example of a run that gave up after too many outside changes:

```
WARNING
apply completed with 2 warnings: group fc-tier1: execution.max_replans_per_run (3) exceeded (last mismatch:
VM 101 is now on 'san-c', not the planned 'san-a') | groups=1 moves_succeeded=2 moves_failed=0
bytes_moved=2199023255552B replans=3 warnings=2 duration=412.6s
warning: group fc-tier1: 9.50 GiB short of its snapshot reserve / free-space requirement after this run's
plan
pve-storage-drs 0.1.7, mode auto, finished 2026-09-20T20:30:00Z
```

- **Line 1** is the state: exactly OK, WARNING or CRITICAL. The plugin turns it into its exit status (0, 1, 2).
- **Line 2** is the summary, naming the first problem if there is one. After the | come label=value performance data your monitoring can graph: groups, moves_succeeded, moves_failed, bytes_moved (bytes), replans, warnings, duration (seconds).
- **The rest** lists any further errors and warnings, one per line (at most 20, then “and N more — see the run log”; a message longer than 300 characters is cut with . . ., since monitoring systems truncate plugin output — the full text is in the run log), and ends with the version, the mode and the finish time. The file holds counts, group and storage names and volume ids — never credentials.

What makes a run OK, WARNING or CRITICAL

State	When
CRITICAL	The run exited non-zero: a move failed; PVE or Prometheus could not be read (for any group, in the first plan or in a re-plan — the run stops at once); or the tool crashed (the file says CRITICAL before the crash propagates, so a bug cannot leave the previous run's OK in place).
WARNING	The run exited 0 but left something for a human: it gave up after <code>execution.max_replans_per_run</code> re-plans because outside changes kept invalidating its plans (try again later); a failed move left a volume on the target, which is reported and never deleted ; a source storage had not released a volume after a move (draining, usually a slow <code>saferemove wipe</code>); or a group is still short of its snapshot reserve or free-space requirement once this run's plan has run — the tool never trades the reserve for balance, so it will not fix that by itself.
OK	None of the above. A run that found nothing worth moving is OK; so is a dry-run.

UNKNOWN is never written by the tool. You will see it from the plugin itself when the file does not exist yet, or is not in the expected shape.

Freshness: why every run rewrites the file

`check_statusfile` judges freshness by the file's **modification time**, not its content: older than `-a` is reported WARNING. So `apply` rewrites the file on **every** run that produced a result — including runs that moved nothing and dry runs — so that a stale file means “the timer stopped”, never “it was a quiet day”.

Two cases deliberately leave the file alone: a run that exited quietly because another instance held the lock (it produced no result, and the file keeps the last real one), and a run that could not read its configuration at all (there is no path to write to; the file simply goes stale). `plan`, `explain`, `show-load` and `--replay` never write it: an operator running them by hand must not overwrite the timer's last result.

The file is written to a temporary file in the same directory and renamed into place, so the plugin can run at any moment without reading half a report. If it cannot be written at all, the run carries on and exits as it would have (a status file that cannot be written must not change what a run did), a `status_file_write_failed` error is logged, and the file turns WARNING by age on its own.

Wiring it into Nagios/NRPE

The check runs on the node that owns `state.json` — the one node the timer runs on:

```
command[check_pve_storage_drs]=usr/lib/nagios/plugins/check_statusfile -a 1h
/var/lib/pve-storage-drs/status
```

Pick `-a` a little longer than the timer's period. Too generous and a stopped timer goes unnoticed for that long; too tight and one slow run turns the check WARNING.